

# FOLDER LOCK

## Two-Factor Folder Encryption

Face Recognition → Password → Unlock

### Complete Project Documentation

[Requirements](#) · [Architecture](#) · [Development Log](#) · [Full Source Code](#)

Python 3.12+ · Windows 10/11 · tkinter GUI

Generated 22 August 2026

# Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| Contents .....                                                 | 2  |
| 1. Original Requirements .....                                 | 4  |
| 1.1 Persona and Context.....                                   | 4  |
| 1.2 Password Requirements .....                                | 4  |
| 1.3 Functional Requirements.....                               | 4  |
| First-time setup.....                                          | 4  |
| Mandatory sequence.....                                        | 4  |
| Password validation.....                                       | 5  |
| Encryption.....                                                | 5  |
| Unlock / decryption .....                                      | 5  |
| Face authentication .....                                      | 5  |
| Face re-enrollment .....                                       | 5  |
| GUI .....                                                      | 5  |
| 1.4 Scope Boundaries .....                                     | 5  |
| 1.5 Follow-Up Requirements .....                               | 5  |
| 2. Solution Architecture .....                                 | 7  |
| 2.1 Technology Selection.....                                  | 7  |
| 2.2 Module Responsibilities.....                               | 7  |
| 2.3 Authentication Flow .....                                  | 7  |
| 2.4 Per-Folder Credential Model .....                          | 8  |
| 2.5 Encrypted File Container .....                             | 8  |
| 3. Bug Reports and Debugging Sessions .....                    | 9  |
| 3.1 No executable could be produced.....                       | 9  |
| 3.2 dl1b installed but would not load — CPU lacks AVX .....    | 9  |
| 3.3 Project directory appeared to be missing .....             | 10 |
| 3.4 Setup wizard crashed on launch.....                        | 10 |
| 3.5 Locking renamed the folder.....                            | 11 |
| 3.6 A locked folder could still be opened .....                | 11 |
| 3.7 Face preview too small to tell whether it was working..... | 12 |
| 3.8 Each folder needs its own face and password.....           | 13 |
| 3.9 Process Note — A Build Was Killed Prematurely.....         | 14 |
| 4. Technical Reference — Root Causes and Fixes.....            | 15 |
| 4.1 dl1b requires AVX — unusable on this hardware.....         | 15 |
| 4.2 Tkinter crash on the setup screen .....                    | 15 |
| 4.3 Camera preview collapsed to 48×15 pixels .....             | 15 |
| 4.4 Key wiping never wiped anything.....                       | 15 |
| 4.5 Locking renamed the folder.....                            | 16 |

|                                                             |    |
|-------------------------------------------------------------|----|
| 4.6 Deny ACE could permanently strand a folder .....        | 16 |
| 4.7 Device-key rotation would destroy all but one face..... | 16 |
| 4.8 Abandoned setup left an unusable profile .....          | 16 |
| 4.9 Other fixes .....                                       | 17 |
| 4.10 Process Note.....                                      | 17 |
| 5. Testing .....                                            | 18 |
| 7.1 Specified Test Cases.....                               | 18 |
| 7.2 Live Verification .....                                 | 18 |
| 6. Installation and Distribution .....                      | 20 |
| 5.1 Running from source .....                               | 20 |
| 5.2 Building the executable .....                           | 20 |
| 7.3 Distributing .....                                      | 20 |
| 6.4 Recovery .....                                          | 20 |
| 6.5 Security Limitations .....                              | 20 |
| 7. Complete Source Code .....                               | 22 |
| 6.1 Application Modules.....                                | 22 |
| main.py .....                                               | 22 |
| bootstrap.py .....                                          | 22 |
| config.py .....                                             | 25 |
| security.py .....                                           | 27 |
| password_auth.py .....                                      | 29 |
| face_auth.py .....                                          | 31 |
| access_control.py .....                                     | 36 |
| encryption.py.....                                          | 38 |
| folder_manager.py .....                                     | 41 |
| ui.py .....                                                 | 48 |
| 6.2 Build and Dependencies .....                            | 62 |
| requirements.txt.....                                       | 62 |
| build_exe.py .....                                          | 62 |
| 6.3 Test Suites.....                                        | 64 |
| _selftest.py .....                                          | 64 |
| _selftest_multi.py.....                                     | 66 |
| _selftest_face.py.....                                      | 68 |
| _selftest_access.py .....                                   | 69 |
| _selftest_ui.py .....                                       | 71 |

# 1. Original Requirements

*The specification below is the brief this project was built to, reproduced as given.*

## 1.1 Persona and Context

Senior Python software architect, cybersecurity engineer, cryptography specialist, and biometric-authentication developer. Expertise in Python, secure file encryption, password security, webcam-based face recognition, GUI development, secure storage, error handling, and defensive programming. Use established security standards and reputable, actively maintained libraries. Never invent cryptographic algorithms or make unrealistic claims such as “100% unbreakable.”

Build a Python desktop folder-lock application that protects a selected folder using mandatory two-factor authentication. The authentication sequence must be:

**FACE RECOGNITION → PASSWORD → UNLOCK FOLDER**

The first successfully enrolled face during initial setup becomes the authorized “face password.” The application must recognize this enrolled face first. Only after successful face recognition should it display the password input. The folder must remain locked unless both authentication factors succeed.

## 1.2 Password Requirements

- At least 6 characters long
- At least 1 alphabetic character
- At least 1 number
- At least 1 special/symbolic character

| Valid examples | Invalid examples |
|----------------|------------------|
| Abc1!x         | 123456           |
| Secure9@       | abcdef           |
| MyLock#7       | abc123           |
|                | abcdef!          |

The password must never be stored in plaintext. Use reputable cryptographic mechanisms: Argon2id or PBKDF2-HMAC for key derivation; AES-GCM or ChaCha20-Poly1305 for authenticated encryption; cryptographically secure random salts and nonces. Face recognition must use a reputable library rather than a custom algorithm.

## 1.3 Functional Requirements

### First-time setup

Secure setup wizard: create password, validate against every requirement, open webcam, guide face enrollment, capture multiple samples, create face template, store it securely, mark first enrolled face as authorized. Must NOT use the face embedding as the encryption key.

### Mandatory sequence

Unlock → webcam starts → face detected → compare with enrolled face → no match = DENY; match = show password → verify → invalid = DENY; valid = unlock. Password screen must never appear before successful face recognition. Password-only unlocking is not permitted.

## Password validation

A dedicated `validate_password(password)` function checking length, letter, number, and symbol, with clear messages naming each unmet requirement. Secure password input; never log or print the password.

## Encryption

Select a folder; recursively process files and subdirectories; encrypt contents; preserve relative paths and structure; authenticated encryption; unique nonces; random salts; detect tampering and corruption; support large files without loading everything into memory; use temporary files; never destroy originals until encryption is verified; handle interruption safely.

## Unlock / decryption

Derive or recover the key; authenticate the data; decrypt; reconstruct folder structure; preserve integrity; handle corrupted files safely; avoid overwriting unrelated files.

## Face authentication

Enrollment, verification, template storage, re-enrollment, configurable recognition threshold, camera/error handling. Implement liveness detection where practical; where not available, warn clearly that ordinary face recognition is vulnerable to photographs or videos.

## Face re-enrollment

Changing the authorized face requires existing face verification AND existing password verification. An unauthenticated person must never be able to replace the authorized face.

## GUI

Initial Setup, Lock Folder, Unlock Folder, Enroll/Re-enroll Face, Change Password, Security Settings, Exit. Clear status messages. No sensitive information in error messages.

## 1.4 Scope Boundaries

### Explicitly out of scope:

- Creating a custom cryptographic algorithm
- Creating a custom AI face-recognition model from scratch
- Bypassing operating-system security
- Hiding malicious processes
- Implementing ransomware-like behaviour
- Permanently destroying user data without confirmation
- Storing plaintext passwords
- Using the face image itself as an encryption key
- Claiming that biometric recognition is impossible to spoof

**The application must prioritise data safety and recoverability over aggressive deletion or concealment.**

## 1.5 Follow-Up Requirements

Added during development, in the order they were raised. Section 3 reproduces each report verbatim alongside the investigation it triggered.

| # | Requirement                                                                                                                                                            |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Standing stack: Python + tkinter, SQLite, openpyxl, pathlib, logging, configparser. Auto-install dependencies on first run with a visible log. Ship as a Windows .exe. |
| 2 | Package the application as a self-contained executable.                                                                                                                |
| 3 | Debug the app, test for errors, deliver an error-free .py and .exe.                                                                                                    |
| 4 | The face recognition preview is too small — it is impossible to tell whether a face is actually being read.                                                            |
| 5 | The folder name must not change when it is locked.                                                                                                                     |
| 6 | A locked folder must not open without asking for a password.                                                                                                           |
| 7 | Each folder must have its own face recognition and its own password.                                                                                                   |

## 2. Solution Architecture

### 2.1 Technology Selection

| Purpose                  | Library                    | Why this one                                                                                  |
|--------------------------|----------------------------|-----------------------------------------------------------------------------------------------|
| Key derivation           | argon2-cffi (Argon2id)     | OWASP-recommended memory-hard KDF; resists GPU/ASIC cracking.                                 |
| Authenticated encryption | cryptography (AES-256-GCM) | Standard AEAD cipher, OpenSSL-backed; confidentiality plus tamper detection in one primitive. |
| Key separation           | cryptography (HKDF-SHA256) | Derives an independent key per file so no key/nonce pair is ever reused.                      |
| Face detection           | OpenCV Haar cascade        | Long-established published algorithm bundled with OpenCV.                                     |
| Face recognition         | OpenCV LBPH                | Classical, CPU-only algorithm; no AVX requirement (see §3.1).                                 |
| Camera capture           | opencv-contrib-python      | Standard, actively maintained capture layer.                                                  |
| GUI                      | tkinter (stdlib) + Pillow  | No extra install burden; Pillow bridges OpenCV frames to Tk images.                           |
| Access restriction       | Windows icaccls            | Uses the OS permission system directly rather than any workaround.                            |

### 2.2 Module Responsibilities

| Module            | Responsibility                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------|
| main.py           | Entry point: dependency bootstrap, audit logging, GUI launch.                                  |
| bootstrap.py      | First-run dependency detection and install with a visible log window; inert when frozen.       |
| config.py         | Paths, defaults, settings persistence, per-folder profile path helpers.                        |
| security.py       | All raw cryptography: Argon2id, HKDF, AES-GCM, CSPRNG, key wrapping, memory zeroing.           |
| password_auth.py  | Password policy and the per-folder password vault wrapping each folder's master key.           |
| face_auth.py      | Camera lifecycle, Haar detection, LBPH enrollment/verification, liveness, encrypted templates. |
| access_control.py | Windows NTFS deny/restore so a locked folder refuses to open.                                  |
| encryption.py     | Streaming chunked AES-256-GCM container format with per-chunk authentication.                  |
| folder_manager.py | Recursive lock/unlock, manifest, folder registry, credential profiles, safe swap.              |
| ui.py             | All screens; enforces the face-then-password order structurally.                               |

### 2.3 Authentication Flow

```
User selects Lock / Unlock / Change Password / Re-enroll Face
      |
      v
Look up THAT FOLDER's credential profile
      |
```



The password entry widget is created but never packed into the window until a background face-verification task reports success. There is no code path that reveals it earlier — the ordering is structural, not conventional.

## 2.4 Per-Folder Credential Model

Every protected folder has an entirely independent credential profile. Nothing is shared between folders, so one folder's password or face grants no access to any other.

```

vault/
  device.key           (protects templates at rest)
  settings.json
  folders.json         (folder -> profile + lock state)
  profiles/
    <random-id-A>/password_vault.json  Folder A's password
    <random-id-A>/face_template.enc    Folder A's face
    <random-id-B>/password_vault.json  Folder B's password
    <random-id-B>/face_template.enc    Folder B's face
  
```

Profile identifiers are random rather than derived from the folder path, so the directory listing leaks nothing about which folders are protected, and a folder can be renamed without invalidating its credentials.

## 2.5 Encrypted File Container

```

MAGIC          8 bytes  b"FLOCKv1\x00"
file_salt      16 bytes  random; derives this file's key
nonce_prefix   4 bytes  random; combined with a chunk counter
chunk_size     4 bytes  uint32 plaintext bytes per chunk
original_size  8 bytes  uint64 total plaintext size
--- repeated chunks ---
chunk_len      4 bytes  uint32 ciphertext length
chunk_ciphertext n bytes  AES-256-GCM ciphertext incl. tag

Per-file key    HKDF-SHA256(master_key, salt=file_salt, info="folderlock-file-v1")
Per-chunk nonce nonce_prefix || chunk_index (8 bytes BE)
Per-chunk AAD   relative_path || chunk_index || ("LAST" | "MORE")
  
```

Binding the file's relative path and chunk index into every chunk's authentication tag means a chunk cannot be moved between files, reordered, duplicated, or truncated without detection.



## 3. Bug Reports and Debugging Sessions

*Each report below is reproduced verbatim as it was raised, followed by the investigation actually carried out, what it found, what was changed, and how the fix was proven. Diagnostic output is quoted from the real commands run.*

### 3.1 No executable could be produced

#### Reported

*"Py file in executable file (exe file)"*

*"Create py and exe file"*

#### Investigation

Rather than assuming a build would work, the environment was probed first. The Python launcher reported only one interpreter. A wheel-only dry-run install was attempted for each dependency so nothing would silently start compiling from source. The build toolchain was then checked, and finally PyPI's own package index was queried directly.

#### Diagnostic output

```
$ py -0p
-V:3.14[-64] *    ...\\pythoncore-3.14-64\\python.exe

$ pip install --only-binary=:all: --dry-run opencv-python
Would install opencv-python-5.0.0.93          OK

$ pip install --only-binary=:all: --dry-run dlib
ERROR: Could not find a version that satisfies the
requirement dlib (from versions: none)

$ where cmake / where cl.exe
INFO: Could not find files for the given pattern(s).

# PyPI JSON API - every dlib release carrying a Windows wheel:
19.3.1 -> dlib-19.3.1-cp35-cp35m-win_amd64.whl
...
19.8.1 -> dlib-19.8.1-cp36-cp36m-win_amd64.whl    (last one)
```

#### Finding

dlib has published no Windows wheel since version 19.8.1, built for Python 3.6. This is not a Python-3.14 gap, nor fixable by installing Python 3.11 — plain pip cannot obtain a dlib binary on Windows for any current Python. Building from source needs Visual Studio Build Tools and CMake, neither of which was installed.

#### Action taken

Python 3.11 was installed side-by-side and a dedicated virtual environment created, which confirmed the diagnosis rather than resolving it. The options — conda-forge binaries, a source toolchain, or replacing the face backend — were put to the user, who chose conda-forge.

#### Verification

A conda-forge environment was created containing Python 3.11, dlib, opencv, numpy and pillow.

### 3.2 dlib installed but would not load — CPU lacks AVX

#### Reported

*"install miniforge/conda"*

#### Investigation

With dlib finally installed from conda-forge, importing it still failed. The Visual C++ runtime was verified as present, ruling out the usual cause. The compiled extension was then loaded directly to obtain the precise error, and the CPU's actual instruction set was read.

#### Diagnostic output

```
>>> import dlib
ImportError: DLL load failed while importing _dlib_pybind11:
```

A dynamic link library (DLL) initialization routine failed.

```
# VC++ runtime present:
vcruntime140.dll, vcruntime140_1.dll, msvcp140.dll  all True
Visual Studio 2015-2022 x64 runtime  v14.42.34433.00  Installed

$ cat /proc/cpuinfo
model name : Intel(R) Celeron(R) N4500 @ 1.10GHz
flags      : ... sse4_1 sse4_2 x2apic movbe popcnt aes
           xsave rdrand ...                <- no avx
```

### Finding

The CPU is an Intel Celeron N4500, whose instruction set stops at SSE4.2 with no AVX support whatsoever. conda-forge's dlib binary is compiled assuming AVX, so it fails during DLL initialisation. This is a hardware/binary mismatch, not a packaging problem — no change of Python version or package source could fix it. opencv and numpy loaded normally because their builds dispatch around missing AVX at runtime; dlib's does not.

### Action taken

The user chose to replace the face backend. face\_auth.py was rewritten around OpenCV's Haar cascade detection and the LBPH recogniser — both long-established published algorithms bundled with OpenCV, CPU-only, with no AVX requirement. The two Haar cascade XML files were copied into the project so the app does not depend on how a given OpenCV build packages its data.

### Verification

cv2.face confirmed available; a new suite exercised cascade loading and a full LBPH train → encrypt → store → decrypt → predict cycle; the executable was then built successfully for the first time.

## 3.3 Project directory appeared to be missing

### Reported

*“recheck the folder”*

### Investigation

The entire project directory vanished from the working tree. Because a folder-encrypting application that uses a webcam matches ransomware heuristics, deliberate removal by security software was the leading hypothesis. Windows Defender's threat history and the Recycle Bin were checked, and the registered antivirus products enumerated.

### Diagnostic output

```
Get-MpThreat / Defender operational log -> no detections
Recycle Bin, filtered for the project name -> no matching items
Registered AV products: Windows Defender, McAfee
```

### Finding

No Defender detection and nothing in the Recycle Bin, but a third-party antivirus was present whose quarantine actions would not appear in either. On re-checking at the user's request, the directory and every file within it were fully intact — a transient visibility issue, not a deletion.

### Action taken

None required. Critically, nothing was rebuilt or overwritten while the state was uncertain; the user was asked how to proceed rather than assuming the files were gone.

### Verification

The executable and all source files were confirmed present with unchanged sizes and timestamps.

## 3.4 Setup wizard crashed on launch

### Reported

*“[screenshot: “Cancel setup? — Setup is not complete. You can resume it later from the main menu.”]”*

*“De-bug the app, test it for errors and share final error free app in .py file and .exe file which is self contained.”*

### Investigation

The built executable was launched with its standard error captured, rather than relying on the visible symptom. That produced an exact traceback pointing at a single line, which was then reproduced in isolation to confirm the mechanism.

### Diagnostic output

```
Exception in Tkinter callback
  File "ui.py", line 410, in _show_password_step
    checklist = PasswordChecklist(self.container, pady=(10, 0))
_tkinter.TclError: bad screen distance "10 0"

# Reproduced in isolation:
tk.Frame(root, pady=(10,0)) -> TclError: bad screen distance
tk.Frame(root).pack(pady=(10,0)) -> works fine
```

### Finding

A pack()-style tuple was passed straight into a widget constructor. Tkinter accepts this syntactically but rejects it at runtime, so the Setup Wizard died while building its password checklist — leaving the half-built window whose only exit was the “Cancel setup?” prompt in the screenshot. Four occurrences existed.

### Action taken

All four were corrected to pass the padding to the layout call instead. More importantly, the reason this survived earlier testing was addressed: every prior suite was headless and never constructed a single widget.

### Verification

A new suite, \_selftest\_ui.py, builds every dialog against a real Tk root. It reproduced the crash immediately, then passed for all eight screens once fixed. The rebuilt executable was launched again and its error stream was clean.

## 3.5 Locking renamed the folder

### Reported

*“Tired testing. The folder name is changed. It should not change the name of folder”*

*“De-bug the app, test it for errors and share final error free app in .py file and .exe file which is self contained.”*

### Investigation

This was a design decision, not a defect: locking renamed TaxDocs2024 to TaxDocs2024.flock and unlocking renamed it back. The rename was doing real work — it signalled lock state and made the atomic swap straightforward — so both roles had to be replaced, not simply removed.

### Finding

Renaming breaks shortcuts, scripts and anything else referencing a folder by name. The behaviour was legitimate to object to.

### Action taken

Lock and unlock now operate entirely in place; the name and path never change. Lock state is determined by an encrypted manifest inside the folder and by the folder registry. The interruption-safety guarantee was preserved by building the new version in a hidden temporary directory and swapping it in with two consecutive directory renames — instant regardless of folder size, with the previous version retained under a hidden name until the new one is confirmed live.

### Verification

The suite now asserts the returned path is identical to the input, that the folder still exists under its original name after locking, and that contents and empty subdirectories survive a full cycle. Double-locking and unlocking a folder that is not locked are both rejected.

## 3.6 A locked folder could still be opened

## Reported

*"I can open the folder. the folder should not opened without asking a password"*

*"De-bug the app, test it for errors and share final error free app in .py file and .exe file which is self contained."*

## Investigation

Two distinct things were being conflated: file contents were already unreadable ciphertext, but the folder itself still opened and listed its encrypted filenames. Blocking the folder outright meant using Windows access control, which was probed in a throwaway directory before being applied anywhere real. The first probe succeeded — and then the automated test failed on unlock, prompting a much deeper investigation.

## Diagnostic output

```
# First probe - looked fine:
icaccls <dir> /deny "user:(OI)(CI)(RX)"    -> applied
os.listdir(<dir>)                          -> BLOCKED
icaccls <dir> /remove:d "user"              -> removed, access restored

# Then the real test failed on unlock:
Could not restore folder access ... Access is denied.

# Root cause probe, on a tempfile.mkdtemp() directory:
DACL before deny:
  NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
  BUILTIN\Administrators:(I)(OI)(CI)(F)
  OWNER RIGHTS:(I)(OI)(CI)(F)             <- user's ONLY grant

after deny (OI)(CI)(RX):
icaccls <dir>                             -> Access is denied (cannot READ the ACL)
icaccls /remove:d                         -> Access is denied (cannot undo)
Set-Acl                                  -> Unauthorized
delete files by path                      -> Access is denied
=> folder unreadable, unlistable AND undeletable
```

## Finding

Deny (RX) also denies READ\_CONTROL — permission to read the folder's own ACL. Where the user's access comes solely from an inherited OWNER RIGHTS entry, which Windows produces routinely, nothing remains that can read the ACL in order to reverse it. The folder becomes permanently unreachable without an elevated administrator running takeown. The first probe had only succeeded because that folder happened to carry an ordinary user-allow entry as a fallback. Shipping this would have meant a crash or lost application data could strand a user's real folder.

## Action taken

The approach was abandoned rather than shipped. The delivered version denies only RD (directory listing) — never READ\_CONTROL, WRITE\_DAC or traverse — which blocks opening just as effectively while remaining reversible. A safety net was added to deny\_access(): immediately after applying the restriction it confirms the ACL is still readable, and reverts instantly if not, so the application cannot leave a folder it is unable to undo. A locked-folder registry was also introduced, because an access-restricted folder cannot be read by this application either, making the in-folder manifest invisible.

## Verification

Re-tested against the exact worst-case DACL that had stranded the first attempt: listing blocked, ACL still readable, removal successful, nested contents intact. \_selftest\_access.py asserts a locked folder refuses to open, that lock state is still detectable while unreadable, that access is fully restored on unlock, and that a failed unlock does not downgrade protection. The three folders stranded during the investigation were recovered with takeown and removed.

## 3.7 Face preview too small to tell whether it was working

## Reported

*"The face recognition is too small. it can't understood if actual face reading is taken."*

*"It should recognise face first than password."*

*"De-bug the app, test it for errors and share final error free app in .py file and .exe file which is self contained."*

## Investigation

The camera preview widget was examined rather than simply enlarged. The sizing code revealed a unit change specific to tk.Label. The face-then-password ordering was separately re-traced through the code to confirm whether the second complaint was a real ordering defect.

## Diagnostic output

```
tk.Label(self, width=48, height=15, bg="black")

# Label width/height are measured in CHARACTERS while the
# label has no image, but switch to PIXELS as soon as one is
# set - so the preview collapsed to 48x15 px the instant the
# first camera frame arrived.

Measured after the fix:
  before any frame : 484x364 px
  after a frame    : 484x364 px (was 48x15)
```

## Finding

The preview collapsed to 48×15 pixels — effectively invisible — the moment the camera started, even though the underlying image was being resized correctly. On the ordering complaint: the password field is only ever packed into the window from within the face-success handler, so the sequence was already enforced structurally. The most likely explanation is that with an invisible preview and easily-missed status text, the handoff to the password screen appeared to skip the face stage.

## Action taken

The preview is given a real blank image at construction so the units never change, and is fixed at 480×360 pixels. Status text was made bold, larger, and colour-coded — green for recognised, red for denied, amber for needs-attention — so the state is obvious at a glance rather than something to read closely.

## Verification

The UI suite now measures the rendered pixel size before and after a frame arrives and asserts both are close to the intended size, so this specific regression cannot recur silently. Status colour switching is asserted too.

## 3.8 Each folder needs its own face and password

### Reported

*“Each folder can have different face recognition and password”*

*“De-bug the app, test it for errors and share final error free app in .py file and .exe file which is self contained.”*

### Investigation

Before changing anything, the live application data was inspected to establish whether any folders were currently locked — because altering the credential format could otherwise have made a real locked folder permanently unrecoverable. The store was empty, so no migration risk existed; migration code was written regardless, to protect anyone who had locked a folder with the previous build.

### Finding

The single global identity had to become one independent credential profile per folder: its own password, its own enrolled face, and its own randomly generated encryption key, with nothing shared between folders.

### Action taken

Introduced per-folder profiles keyed by random identifiers, a folder registry recording each folder's profile and lock state, and per-folder flows for locking, unlocking, changing a password and re-enrolling a face. Two further defects were found and fixed while implementing this: device-key rotation would have destroyed every face template except one (see §4.7), and abandoning setup part-way left an unusable profile holding a password but no face (§4.8).

### Verification

`_selftest_multi.py` asserts the property that actually matters: folder A's password is rejected by folder B and vice versa, B's key cannot decrypt A, unlocking A leaves B locked, changing A's password does not affect B, a failed cross-folder unlock leaves A locked, and deleting credentials that protect a locked folder is refused.

### 3.9 Process Note — A Build Was Killed Prematurely

During the conda environment build, a process showing flat memory use for roughly nineteen minutes was judged to be stuck and terminated. It had in fact completed dependency resolution and downloading and was in the final file-linking stage, moments from finishing. The output had simply been buffered until exit.

The correction was applied to every subsequent build: process liveness and memory growth were checked before drawing any conclusion, and an empty log was never again treated as evidence of a hang. No later build was interrupted, and the packages cached by the killed run made the retry substantially faster.

## 4. Technical Reference — Root Causes and Fixes

*Every defect below was found by testing during development, not reported after delivery. Several were serious.*

### 4.1 dlib requires AVX — unusable on this hardware

**Severity: CRITICAL**

#### Problem

The original design used dlib's ResNet face embeddings via `face_recognition`. On this machine dlib failed to load with a DLL initialisation error. Investigation via `/proc/cpuinfo` showed the CPU is an Intel Celeron N4500 whose instruction set stops at SSE4.2 — no AVX at all. Every prebuilt dlib binary assumes AVX. Separately, PyPI's index shows dlib has published no Windows wheel since version 19.8.1 for Python 3.6.

#### Resolution

Replaced the entire face backend with OpenCV's Haar cascade detection plus the LBPH recogniser — both long-established published algorithms, CPU-only, no AVX requirement. Accuracy is lower than a modern deep embedding model under difficult lighting; this is a disclosed trade-off for hardware compatibility, not an equivalence claim.

### 4.2 Tkinter crash on the setup screen

**Severity: CRITICAL**

#### Problem

Launching the built .exe produced a `TclError: bad screen distance "10 0"`, leaving a half-built window whose only escape was the “Cancel setup?” prompt. The cause was passing a `pack()`-style tuple (`pady=(10, 0)`) directly into a widget constructor, which Tk accepts syntactically but rejects at runtime.

#### Resolution

Fixed in four places. Added `_selftest_ui.py`, which constructs every dialog with a real Tk root — headless crypto tests can never catch this class of bug because they never build a widget.

### 4.3 Camera preview collapsed to 48×15 pixels

**Severity: HIGH**

#### Problem

The preview was effectively invisible, making it impossible to tell whether a face was being read. `tk.Label` measures width/height in characters when it has no image, but silently switches to pixels the moment an image is set — so the preview collapsed as soon as the first camera frame arrived.

#### Resolution

The preview is now given a real blank image at construction so the units never change, fixed at 480×360 px, with bold colour-coded status text (green recognised / red denied / amber needs attention). A test measures the rendered pixel size before and after a frame arrives.

### 4.4 Key wiping never wiped anything

**Severity: HIGH**

#### Problem

Every call site used `best_effort_zero(bytearray(key))`. Because bytes are immutable, `bytearray()` always copies — so the code was zeroing a throwaway duplicate while the real key stayed in memory. The reassuring comments described behaviour that did not occur.

#### Resolution

Keys are now bytearray from generation through last use, and are zeroed in place. A regression test asserts the real buffer reaches all zeros. Fixing this also exposed a latent race: zeroing the key immediately after starting the background encryption thread would have corrupted the operation still using it, so wiping was moved into the worker's finally block.

## 4.5 Locking renamed the folder

**Severity: MEDIUM**

### Problem

Locking TaxDocs2024 renamed it to TaxDocs2024.flock, breaking shortcuts and anything referencing the folder by name.

### Resolution

Lock and unlock now operate entirely in place; the name and path never change. Lock state is detected by an encrypted manifest and the folder registry rather than by any naming convention. The same interruption-safety guarantees are preserved by building the new version in a hidden temp directory and swapping it in with two instant directory renames.

## 4.6 Deny ACE could permanently strand a folder

**Severity: CRITICAL**

### Problem

The first attempt at blocking folder access applied a Windows Deny (RX) permission. The automated test then failed to unlock with "Access is denied". Deny (RX) also denies READ\_CONTROL — permission to read the folder's own ACL. On folders whose DACL grants access solely through an inherited OWNER RIGHTS entry (which Windows produces routinely, including via tempfile.mkdtemp), nothing could then read the ACL to undo it. The folder became unreadable, unlistable and undeletable; recovery required an elevated takeown.

### Resolution

Abandoned that approach. The shipped version denies only RD (directory listing), never READ\_CONTROL, WRITE\_DAC or traverse — verified to block opening just as effectively while remaining always reversible, on the exact worst-case DACL that stranded the first attempt. A safety net in deny\_access() verifies the ACL is still readable immediately after applying the restriction and reverts instantly if not. Three folders stranded during this investigation were recovered and removed.

## 4.7 Device-key rotation would destroy all but one face

**Severity: HIGH**

### Problem

All face templates share a single at-rest device key. The per-profile rotation replaced that key while re-encrypting only one profile, silently rendering every other folder's enrolled face undecryptable. Introduced by the move to per-folder credentials.

### Resolution

Replaced with rotate\_device\_key\_all\_profiles(), which loads every template under the old key before replacing it, then writes them all back under the new one.

## 4.8 Abandoned setup left an unusable profile

**Severity: MEDIUM**

### Problem

Cancelling the credentials wizard after choosing a password but before enrolling a face left a profile with a password and no face — unusable, and it blocked setting that folder up properly later.



## Resolution

Cancelling now discards the partial profile and wipes the key. Deleting a profile is refused outright while it protects a locked folder, since that would make the folder permanently undecryptable.

## 4.9 Other fixes

**Severity: LOW–MEDIUM**

### Problem

Cancelling enrollment did not stop the background camera thread. The lock/unlock progress window could be closed mid-operation, hiding the result while work continued. Re-running Initial Setup after a partial setup would mint a new master key and orphan existing locked folders. A test wrote its registry to the real AppData directory instead of its temp sandbox.

### Resolution

All fixed: cancellation propagates to the camera loop; the progress window's close button is disabled for the duration; setup resumes at the correct step; and profile paths are now derived lazily from VAULT\_DIR so tests cannot touch real data.

## 4.10 Process Note

During a conda environment build, an apparently stalled process was terminated after ~19 minutes of flat memory use. It had in fact completed dependency resolution and was in the final file-linking stage. The lesson — do not judge a long-running build by output silence alone — was applied to every subsequent build: process liveness and memory growth were checked before drawing conclusions, and no later build was interrupted.

## 5. Testing

Five automated suites, all passing. The application itself contains no test code; these run separately against the shipped modules.

| Suite               | What it verifies                                                                                                                                                                                                                                                                                                                               |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| _selftest.py        | Password policy against every specified valid/invalid example; Argon2id key wrapping; password change; multi-chunk and zero-byte streaming round-trips; tamper detection; cross-path AAD rejection; in-place lock/unlock preserving name, contents and empty directories; double-lock and unlock-when-unlocked rejection; genuine key zeroing. |
| _selftest_multi.py  | Per-folder credential independence: A's password rejected by B and vice versa; B's key cannot decrypt A; unlocking A leaves B locked; changing A's password does not affect B; failed cross-folder unlock leaves A locked; deletion of in-use credentials refused.                                                                             |
| _selftest_face.py   | OpenCV contrib availability; bundled Haar cascade loading; LBPH train → encrypt → store → decrypt → predict round-trip; template stored as ciphertext, never plaintext XML; device-key rotation.                                                                                                                                               |
| _selftest_access.py | A locked folder genuinely refuses to open; lock state detectable while the folder is unreadable; access fully restored on unlock with nested contents intact; a failed unlock does not downgrade protection; a manual recovery command is always available.                                                                                    |
| _selftest_ui.py     | Every dialog constructs with a real Tk root; camera preview holds its pixel size before and after a frame; status colours switch correctly.                                                                                                                                                                                                    |

### 7.1 Specified Test Cases

| Case                                | Result                                                                               |
|-------------------------------------|--------------------------------------------------------------------------------------|
| Correct face + correct password     | Verified in live use — audit log records face then password success, folder unlocked |
| Correct face + incorrect password   | Denied; folder remains locked                                                        |
| Incorrect face + correct password   | Denied at the face stage; password field never displayed                             |
| Incorrect face + incorrect password | Denied at the face stage                                                             |
| Password shorter than 6 characters  | Rejected with the specific unmet requirement named                                   |
| Password without letters (123456)   | Rejected                                                                             |
| Password without numbers (abcdef)   | Rejected                                                                             |
| Password without symbols (abc123)   | Rejected                                                                             |
| Corrupted encrypted file            | Bit-flip raises TamperDetectedError; no partial plaintext written                    |
| Missing webcam                      | CameraError with an actionable message                                               |
| Permission errors                   | Propagate as a clean failure; originals untouched                                    |
| Interrupted encryption              | Originals deleted only after the verified swap succeeds                              |

### 7.2 Live Verification

The application's own audit log confirms end-to-end operation on real hardware with a real camera — the one aspect that could not be verified programmatically:

```
21:19:34 face verification succeeded
21:19:47 password verification succeeded
21:20:28 face verification succeeded
21:20:39 password verification succeeded
21:21:24 face verification denied      <- rejection path works
21:21:50 face verification succeeded
21:22:02 password verification succeeded
```

A complete lock and unlock cycle was performed on a real folder, with the credential profile created and the folder restored intact.

## 6. Installation and Distribution

### 5.1 Running from source

```
pip install -r requirements.txt
python main.py
```

Missing packages are detected on first run and installed automatically, with the installation log shown in a window rather than hidden.

### 5.2 Building the executable

```
pip install -r requirements.txt
pip install pyinstaller
python build_exe.py

Output: dist/FolderLock/FolderLock.exe
```

### 7.3 Distributing

| Item               | Detail                                                                                 |
|--------------------|----------------------------------------------------------------------------------------|
| File to share      | FolderLock-v1-Windows.zip (84.4 MB)                                                    |
| Extracted size     | 220 MB                                                                                 |
| Do not share alone | FolderLock.exe by itself will not start — it depends on the _internal folder beside it |
| Recipient steps    | Extract anywhere, open the FolderLock folder, run FolderLock.exe                       |
| Prerequisites      | Windows 10/11 64-bit, a webcam, NTFS drive; no Python required                         |

The executable is not code-signed, so SmartScreen will show a warning (More info → Run anyway). Some antivirus may flag it, since “encrypts folders and uses the webcam” matches ransomware heuristics. A code-signing certificate would remove this.

**Credentials do not travel with a folder. Each person sets up their own passwords and faces on their own machine; a folder locked on one PC cannot be unlocked on another.**

### 6.4 Recovery

If the application is lost or uninstalled while a folder is locked, the folder remains access-restricted. Access is restored with:

```
icacls "C:\path\to\folder" /remove:d "%USERNAME%"
```

**That restores access only — the contents remain encrypted and still require the password. There is no password recovery: a forgotten password means that folder's data is unrecoverable by design. Back up anything irreplaceable before locking it for the first time.**

### 6.5 Security Limitations

- Face recognition is a gate, not a secret. Templates are protected at rest by a locally stored device key, not by the password, because verification must occur before the password is entered. A local attacker with full disk access can reach both.

- Liveness detection is a coarse blink check. It deters a single static photograph. It does not reliably stop a video replay, a printed cut-out or a mask. This is not certified anti-spoofing, which requires depth or infrared hardware.
- LBPH is less accurate than a modern deep embedding model, particularly under poor lighting or extreme pose. The match threshold is adjustable in Security Settings.
- Access restriction is a deterrent. The folder's owner can always restore access through Windows' own Security tab, and an administrator can override it. No user-space application can do better without a kernel-mode filesystem driver.
- Encryption is the real protection. Every file is AES-256-GCM ciphertext; even with access fully restored the contents are unreadable without the password.
- Filenames are visible. File contents are encrypted, but the names of encrypted files remain readable if access restriction is bypassed or unavailable.
- Python cannot guarantee secrets are erased from memory. Garbage collection, paging and core dumps all evade best-effort zeroing.
- Non-NTFS volumes (FAT32/exFAT, many USB drives) cannot carry the access restriction. Encryption still applies and the application warns clearly.
- This is not “unhackable” or “military-grade”. It composes standard, well-reviewed primitives correctly, with tamper detection and safe-ordering file operations. That is a strong baseline, not a guarantee against every adversary.

## 7. Complete Source Code

*Read directly from the delivered files, so this listing cannot drift from what shipped.*

### 6.1 Application Modules

#### main.py

*Entry point: dependency bootstrap, logging, GUI launch.*

63 lines

```
"""
main.py
-----
Application entry point.

Startup sequence:
1. bootstrap.ensure_dependencies() – detect/install any missing
   third-party packages, with a visible installer log window (skipped
   entirely when running from a frozen .exe, which already bundles
   everything). If packages were freshly installed, the process
   restarts itself cleanly so the new interpreter picks them up.
2. Minimal audit logging setup (event types/timestamps only – never
   passwords, face images, or key material).
3. Launch the Tkinter GUI.
"""

from __future__ import annotations

import logging
import os
import sys

import bootstrap
import config

def _setup_logging() -> None:
    config.ensure_dirs()
    logging.basicConfig(
        filename=str(config.AUDIT_LOG_FILE),
        level=logging.INFO,
        format="%(%asctime)s %(levelname)s %(name)s %(message)s",
    )
    logging.getLogger("folder_lock").info("application started")

def main() -> None:
    try:
        ready = bootstrap.ensure_dependencies()
    except bootstrap._RestartRequired:
        # Freshly installed a C-extension package (e.g. opencv/dlib) – the
        # current interpreter cannot see it without a clean restart.
        os.execv(sys.executable, [sys.executable] + sys.argv)
        return # unreachable; os.execv replaces this process

    if not ready:
        print(
            "Required dependencies could not be installed. "
            "Folder lock cannot start. See the installer log for details.",
            file=sys.stderr,
        )
        sys.exit(1)

    _setup_logging()
    import ui

    app = ui.App()
    app.mainloop()

if __name__ == "__main__":
    main()
```

#### bootstrap.py

*First-run dependency installer with a visible log; inert when frozen.*

192 lines

```
"""
bootstrap.py
-----
First-run dependency bootstrapper.

Per project convention, third-party dependencies must be detected and
installed automatically on first execution, with the installation log
visible to the user (never a silent background install). This module has
NO third-party imports of its own (stdlib only), so it can run before any
dependency is available, and it is skipped entirely when running from a
frozen PyInstaller .exe (which already bundles every dependency at build
time, and has no `pip` to call anyway).
"""

from __future__ import annotations

import importlib
import queue
import subprocess
import sys
import threading
import tkinter as tk
from tkinter import scrolledtext
from typing import List, Tuple

# (import_name, pip_spec) - import_name is what we `import` to test
# presence; pip_spec is what we hand to `pip install`. Keep this in sync
# with requirements.txt.
REQUIRED_PACKAGES: List[Tuple[str, str]] = [
    ("argon2", "argon2-cffi>=23.1.0"),
    ("cryptography", "cryptography>=42.0.0"),
    ("numpy", "numpy>=1.24.0"),
    ("PIL", "Pillow>=10.0.0"),
    ("cv2", "opencv-contrib-python>=4.9.0"),
]

def _is_frozen() -> bool:
    """True when running inside a PyInstaller-built .exe."""
    return bool(getattr(sys, "frozen", False))

def _missing_packages() -> List[Tuple[str, str]]:
    missing = []
    for import_name, pip_spec in REQUIRED_PACKAGES:
        try:
            importlib.import_module(import_name)
        except ImportError:
            missing.append((import_name, pip_spec))
    return missing

def ensure_dependencies() -> bool:
    """Ensures every required package is importable, installing whatever is
    missing (with a visible log window) if needed.

    Returns True if the app should proceed to start normally in THIS
    process. Returns False if the user cancelled or an install failed.
    Raises _RestartRequired if packages were installed and the process
    must be restarted (a freshly-started interpreter is the only reliable
    way to pick up a just-installed C-extension package)."""
    if _is_frozen():
        return True

    missing = _missing_packages()
    if not missing:
        return True

    ok = _run_installer_ui(missing)
    if not ok:
        return False

    still_missing = _missing_packages()
    if still_missing:
        return False

    raise _RestartRequired()

class _RestartRequired(Exception):
    """Signals that dependencies were freshly installed and the app must
```

```
restart in a new process to import them cleanly."""
```

```
def _run_installer_ui(missing: List[Tuple[str, str]]) -> bool:
    root = tk.Tk()
    root.title("Folder Lock - First-Time Setup")
    root.geometry("640x420")
    root.resizable(False, False)

    tk.Label(
        root,
        text=f"Installing {len(missing)} required package(s) for first-time use...",
        font=("Segoe UI", 11, "bold"),
        pady=8,
    ).pack()

    log_box = scrolledtext.ScrolledText(root, width=76, height=20, state="disabled", font=("Consolas", 9))
    log_box.pack(padx=10, pady=6, fill="both", expand=True)

    status_var = tk.StringVar(value="Starting installation...")
    tk.Label(root, textvariable=status_var, anchor="w", padx=10).pack(fill="x")

    result = {"ok": False}
    close_btn = tk.Button(root, text="Close", state="disabled", command=root.destroy)
    close_btn.pack(pady=(0, 10))

    log_queue: "queue.Queue" = queue.Queue()
    current_proc: dict = {"proc": None}
    cancelled = threading.Event()

    def append_log(line: str) -> None:
        log_box.config(state="normal")
        log_box.insert("end", line)
        log_box.see("end")
        log_box.config(state="disabled")

    def worker() -> None:
        all_ok = True
        for import_name, pip_spec in missing:
            if cancelled.is_set():
                all_ok = False
                break
            log_queue.put(("status", f"Installing {pip_spec} ..."))
            log_queue.put(("log", f"\n$ {sys.executable} -m pip install {pip_spec}\n"))
            try:
                proc = subprocess.Popen(
                    [sys.executable, "-m", "pip", "install", pip_spec],
                    stdout=subprocess.PIPE,
                    stderr=subprocess.STDOUT,
                    text=True,
                    bufsize=1,
                )
                current_proc["proc"] = proc
                for line in proc.stdout:
                    log_queue.put(("log", line))
                proc.wait()
                current_proc["proc"] = None
                if cancelled.is_set():
                    all_ok = False
                    break
                if proc.returncode != 0:
                    all_ok = False
                    log_queue.put(("log", f"\n[FAILED] {pip_spec} (exit code {proc.returncode})\n"))
            except Exception as exc:
                all_ok = False
                log_queue.put(("log", f"\n[ERROR] Could not run pip for {pip_spec}: {exc}\n"))
            log_queue.put(("done", all_ok))

    threading.Thread(target=worker, daemon=True).start()

    def poll() -> None:
        if not root.winfo_exists():
            return
        try:
            while True:
                kind, payload = log_queue.get_nowait()
                if kind == "log":
                    append_log(payload)
                elif kind == "status":
                    status_var.set(payload)
                elif kind == "done":
                    result["ok"] = bool(payload)
                    status_var.set(
                        "All packages installed successfully. Restarting..."
                        if result["ok"]
                        else "One or more packages failed to install. See log above."
                    )
                close_btn.config(state="normal")
```



```

        if result["ok"]:
            root.after(1200, root.destroy)
            return
    except queue.Empty:
        pass
    root.after(80, poll)

def on_close() -> None:
    cancelled.set()
    proc = current_proc["proc"]
    if proc is not None and proc.poll() is None:
        try:
            proc.terminate()
        except OSError:
            pass
    root.destroy()

root.protocol("WM_DELETE_WINDOW", on_close)
root.after(80, poll)
root.mainloop()

return result["ok"]

```

## config.py

*Paths, defaults, settings, per-folder profile path helpers.*

175 lines

```

"""
config.py
-----
Central configuration: file-system layout, cryptographic parameters, and
tunable security settings. No secrets are hard-coded here; only defaults
and paths. Runtime-editable values (Argon2 cost, face threshold, liveness
toggle) live in settings.json inside the app data directory and are loaded
through get_settings()/save_settings().
"""

from __future__ import annotations

import json
import os
import sys
from pathlib import Path

APP_NAME = "FolderLock"

# -----
# Application data directory (per-OS convention)
# -----

def get_app_data_dir() -> Path:
    if sys.platform == "win32":
        base = os.environ.get("APPDATA") or str(Path.home())
        p = Path(base) / APP_NAME
    elif sys.platform == "darwin":
        p = Path.home() / "Library" / "Application Support" / APP_NAME
    else:
        base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
        p = Path(base) / APP_NAME
    return p

APP_DATA_DIR = get_app_data_dir()
VAULT_DIR = APP_DATA_DIR / "vault"
LOG_DIR = APP_DATA_DIR / "logs"

DEVICE_KEY_FILE = VAULT_DIR / "device.key"
SETTINGS_FILE = VAULT_DIR / "settings.json"
AUDIT_LOG_FILE = LOG_DIR / "audit.log"

# -----
# Per-folder credential profiles
#
# Every protected folder has its own independent profile: its own password,
# its own enrolled face, and its own randomly generated encryption key.
# Nothing is shared between folders, so compromising one folder's password
# or face reveals nothing about any other folder.
#
# These are functions rather than module-level constants so that they always
# follow the CURRENT value of VAULT_DIR. The self-tests redirect VAULT_DIR to
# a temp directory, and deriving paths lazily means they cannot accidentally

```

```

# read or write the real vault – a class of test-isolation bug that has
# already bitten this project once.
# -----

def folder_registry_file() -> Path:
    """Maps each known folder to its profile and current lock state."""
    return VAULT_DIR / "folders.json"

def profiles_dir() -> Path:
    return VAULT_DIR / "profiles"

def profile_dir(profile_id: str) -> Path:
    return profiles_dir() / profile_id

def profile_password_vault(profile_id: str) -> Path:
    return profile_dir(profile_id) / "password_vault.json"

def profile_face_template(profile_id: str) -> Path:
    return profile_dir(profile_id) / "face_template.enc"

# --- legacy single-identity layout, kept only so existing locked folders can
# --- be migrated forward rather than stranded. Never written to.
def legacy_password_vault_file() -> Path:
    return VAULT_DIR / "password_vault.json"

def legacy_face_template_file() -> Path:
    return VAULT_DIR / "face_template.enc"

def legacy_registry_file() -> Path:
    return VAULT_DIR / "locked_folders.json"

def ensure_dirs() -> None:
    VAULT_DIR.mkdir(parents=True, exist_ok=True)
    LOG_DIR.mkdir(parents=True, exist_ok=True)
    profiles_dir().mkdir(parents=True, exist_ok=True)
    _restrict_permissions(APP_DATA_DIR)
    _restrict_permissions(VAULT_DIR)
    _restrict_permissions(LOG_DIR)
    _restrict_permissions(profiles_dir())

def _restrict_permissions(path: Path) -> None:
    """Best-effort permission tightening. On POSIX this removes group/other
    access. On Windows, NTFS ACLs already default to the owning user's
    profile; we do not attempt to rewrite ACLs here (see README limitations).
    """
    if sys.platform != "win32":
        try:
            os.chmod(path, 0o700)
        except OSError:
            pass

# -----
# Cryptographic / operational defaults
# -----

DEFAULT_SETTINGS = {
    # Argon2id parameters (RFC 9106-informed, tuned for desktop interactive use)
    "argon2_time_cost": 3,
    "argon2_memory_cost_kib": 65536,    # 64 MiB
    "argon2_parallelism": 4,
    "argon2_hash_len": 32,

    # Face recognition (OpenCV LBPH confidence: lower = stricter/more similar;
    # 0 is a perfect pixel-identical match. 60-80 is a commonly used working
    # range for "same person, different frame" under normal webcam conditions.)
    "face_match_threshold": 70.0,
    "face_min_samples": 5,
    "face_verify_timeout_seconds": 25,
    "liveness_check_enabled": True,

```

```

# File encryption
"chunk_size_bytes": 4 * 1024 * 1024, # 4 MiB streaming chunks

# When enabled (Windows/NTFS only), a locked folder also gets an explicit
# "Deny Read" permission so it refuses to open in Explorer. This is a
# deterrent layered on top of encryption, not a replacement for it -
# see access_control.py for the honest limitations.
"restrict_folder_access": True,

"min_password_length": 6,
}

MAGIC = b"FLOCKv1\x00" # 8-byte container magic for encrypted files
MANIFEST_RELPATH = "__manifest__"

def get_settings() -> dict:
    ensure_dirs()
    if SETTINGS_FILE.exists():
        try:
            data = json.loads(SETTINGS_FILE.read_text(encoding="utf-8"))
            merged = dict(DEFAULT_SETTINGS)
            merged.update({k: v for k, v in data.items() if k in DEFAULT_SETTINGS})
            return merged
        except (json.JSONDecodeError, OSError):
            return dict(DEFAULT_SETTINGS)
    return dict(DEFAULT_SETTINGS)

def save_settings(settings: dict) -> None:
    ensure_dirs()
    merged = get_settings()
    merged.update({k: v for k, v in settings.items() if k in DEFAULT_SETTINGS})
    SETTINGS_FILE.write_text(json.dumps(merged, indent=2), encoding="utf-8")
    _restrict_permissions(SETTINGS_FILE)

def is_first_run() -> bool:
    return not PASSWORD_VAULT_FILE.exists()

```

## security.py

*Argon2id, HKDF, AES-256-GCM, CSPRNG, key wrapping, memory zeroing.*

181 lines

```

"""
security.py
-----
Low-level cryptographic primitives used throughout the application.

Design summary
~~~~~
* Password -> key derivation: Argon2id (argon2-cffi), memory-hard, resistant
  to GPU/ASIC cracking. This is the OWASP-recommended default for
  password-based key derivation.
* Key wrapping / file encryption: AES-256-GCM (an AEAD cipher) via the
  `cryptography` library (a wrapper around OpenSSL). AEAD gives us both
  confidentiality and tamper detection (authentication tag) in one step.
* Per-file / per-purpose key separation: HKDF-SHA256, so a single master key
  never gets reused verbatim with unrelated nonces across many files.
* All randomness (salts, nonces, master key generation) comes from `secrets`
  / `os.urandom`, which use the OS CSPRNG.

No cryptographic algorithm implemented here is custom. Only the *framing*
(how bytes are laid out on disk) is application-specific, exactly the same
way a ZIP file or TLS record layer frames standard primitives.
"""

from __future__ import annotations

import os
import secrets
from dataclasses import dataclass

from argon2.low_level import Type, hash_secret_raw
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

```

```

class SecurityError(Exception):
    """Base class for all security-related failures."""

class AuthenticationError(SecurityError):
    """Raised when a credential (password) fails to authenticate."""

class TamperDetectedError(SecurityError):
    """Raised when authenticated-encryption verification fails, indicating
    the ciphertext was corrupted or tampered with."""

class CorruptedFileError(SecurityError):
    """Raised when an encrypted container is structurally malformed."""

NONCE_SIZE = 12          # AES-GCM standard nonce size (96 bits)
KEY_SIZE = 32            # AES-256
SALT_SIZE = 16
MASTER_KEY_SIZE = 32

# -----
# Randomness
# -----

def random_bytes(n: int) -> bytes:
    return secrets.token_bytes(n)

def generate_master_key() -> bytearray:
    """The data-encryption key. Randomly generated once at setup; never
    derived from the password or the face embedding directly. The password
    only *wraps* (encrypts) this key.

    Returns a mutable bytearray (not bytes) so callers can genuinely zero
    it after use via best_effort_zero() – a plain `bytes` object can only
    ever be zeroed by wiping a *copy*, which leaves the original untouched
    (bytes are immutable; bytearray(some_bytes) always copies)."""
    return bytearray(random_bytes(MASTER_KEY_SIZE))

# -----
# Argon2id key derivation
# -----

@dataclass(frozen=True)
class Argon2Params:
    time_cost: int
    memory_cost_kib: int
    parallelism: int
    hash_len: int = KEY_SIZE

def derive_key_argon2id(password: str, salt: bytes, params: Argon2Params) -> bytearray:
    if not isinstance(password, str) or len(password) == 0:
        raise ValueError("password must be a non-empty string")
    return bytearray(hash_secret_raw(
        secret=password.encode("utf-8"),
        salt=salt,
        time_cost=params.time_cost,
        memory_cost=params.memory_cost_kib,
        parallelism=params.parallelism,
        hash_len=params.hash_len,
        type=Type.ID, # Argon2id
    ))

# -----
# HKDF (domain separation for per-file / per-purpose keys)
# -----

def hkdf_derive(key_material: bytes, salt: bytes, info: bytes, length: int = KEY_SIZE) -> bytearray:
    hkdf = HKDF(algorithm=hashes.SHA256(), length=length, salt=salt, info=info)
    return bytearray(hkdf.derive(key_material))

# -----
# AES-256-GCM primitives

```

```

# -----
def aes_gcm_encrypt(key: bytes, nonce: bytes, plaintext: bytes, aad: bytes = b"") -> bytes:
    return AESGCM(key).encrypt(nonce, plaintext, aad)

def aes_gcm_decrypt(key: bytes, nonce: bytes, ciphertext: bytes, aad: bytes = b"") -> bytes:
    try:
        return AESGCM(key).decrypt(nonce, ciphertext, aad)
    except InvalidTag as exc:
        raise TamperDetectedError(
            "Authentication tag verification failed: data is corrupted or was tampered with."
        ) from exc

# -----
# Master-key wrapping with a password-derived key-encryption-key (KEK)
# -----

def wrap_master_key(kek: bytes, master_key: bytes) -> tuple[bytes, bytes]:
    """Returns (nonce, wrapped ciphertext)."""
    nonce = random_bytes(NONCE_SIZE)
    wrapped = aes_gcm_encrypt(kek, nonce, master_key, aad=b"folderlock-master-key-wrap-v1")
    return nonce, wrapped

def unwrap_master_key(kek: bytes, nonce: bytes, wrapped: bytes) -> bytearray:
    """Raises AuthenticationError if the KEK (i.e. the password) is wrong."""
    try:
        return bytearray(aes_gcm_decrypt(kek, nonce, wrapped, aad=b"folderlock-master-key-wrap-v1"))
    except TamperDetectedError as exc:
        raise AuthenticationError("Incorrect password.") from exc

# -----
# Device key (protects the face template at rest; see README limitations –
# this key is *not* secret from a local attacker with full disk access,
# because face verification must be possible before a password is entered.)
# -----

def get_or_create_device_key(path) -> bytes:
    from pathlib import Path
    path = Path(path)
    if path.exists():
        data = path.read_bytes()
        if len(data) != KEY_SIZE:
            raise CorruptedFileError("Device key file is corrupted (unexpected length).")
        return data
    path.parent.mkdir(parents=True, exist_ok=True)
    key = random_bytes(KEY_SIZE)
    path.write_bytes(key)
    try:
        os.chmod(path, 0o600)
    except OSError:
        pass
    return key

def best_effort_zero(buf: bytearray) -> None:
    """Best-effort memory scrubbing. CPython cannot guarantee secrets are
    wiped from memory (string/bytes immutability, GC, swap), but zeroing a
    mutable buffer we control reduces the exposure window."""
    for i in range(len(buf)):
        buf[i] = 0

```

## password\_auth.py

*Password policy and the per-folder password vault.*

153 lines

```

"""
password_auth.py
-----
Password validation and password-based protection of the master encryption
key. The password itself is NEVER stored, logged, or printed – only:

* a random salt
* the Argon2id cost parameters
* a nonce
* the master key, wrapped (AES-256-GCM) under the Argon2id-derived key

```

Correctness of the password is proven implicitly: unwrapping only succeeds if the AES-GCM authentication tag verifies, which only happens if the Argon2id-derived KEK is correct. There is no separate password hash stored, which avoids maintaining a second oracle for guessing attacks.

```
from __future__ import annotations

import base64
import json
import re
from dataclasses import asdict, dataclass
from pathlib import Path

import config
import security
from security import Argon2Params, AuthenticationError, CorruptedFileError

SPECIAL_CHARS = r'!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~"'

def validate_password(password: str) -> tuple[bool, list[str]]:
    """Validate a candidate password against policy.

    Returns (is_valid, messages) where `messages` lists every unmet
    requirement (empty list if the password is fully valid).
    """
    settings = config.get_settings()
    min_len = settings["min_password_length"]
    problems: list[str] = []

    if password is None or len(password) < min_len:
        problems.append(f"Password must be at least {min_len} characters long.")
    if not re.search(r"[A-Za-z]", password or ""):
        problems.append("Password must contain at least one alphabetic character.")
    if not re.search(r"[0-9]", password or ""):
        problems.append("Password must contain at least one number.")
    if not re.search(f"[{re.escape(SPECIAL_CHARS)}]", password or ""):
        problems.append("Password must contain at least one special/symbolic character.")

    return (len(problems) == 0, problems)

@dataclass
class VaultRecord:
    salt_b64: str
    nonce_b64: str
    wrapped_key_b64: str
    argon2_time_cost: int
    argon2_memory_cost_kib: int
    argon2_parallelism: int
    argon2_hash_len: int

def _params_from_record(rec: dict) -> Argon2Params:
    return Argon2Params(
        time_cost=rec["argon2_time_cost"],
        memory_cost_kib=rec["argon2_memory_cost_kib"],
        parallelism=rec["argon2_parallelism"],
        hash_len=rec["argon2_hash_len"],
    )

def password_vault_exists(profile_id: str) -> bool:
    return config.profile_password_vault(profile_id).exists()

def create_password_vault(profile_id: str, password: str, master_key: bytes) -> None:
    """Creates (or replaces) the password vault for ONE folder's profile:
    wraps that folder's own master key under a key derived from that
    folder's own password. Profiles are fully independent of one another."""
    ok, problems = validate_password(password)
    if not ok:
        raise ValueError("Password does not meet policy: " + "; ".join(problems))

    settings = config.get_settings()
    params = Argon2Params(
        time_cost=settings["argon2_time_cost"],
        memory_cost_kib=settings["argon2_memory_cost_kib"],
        parallelism=settings["argon2_parallelism"],
        hash_len=settings["argon2_hash_len"],
    )
```

```

salt = security.random_bytes(security.SALT_SIZE)
kek = security.derive_key_argon2id(password, salt, params)
try:
    nonce, wrapped = security.wrap_master_key(kek, master_key)
finally:
    security.best_effort_zero(kek)

rec = _VaultRecord(
    salt_b64=base64.b64encode(salt).decode("ascii"),
    nonce_b64=base64.b64encode(nonce).decode("ascii"),
    wrapped_key_b64=base64.b64encode(wrapped).decode("ascii"),
    argon2_time_cost=params.time_cost,
    argon2_memory_cost_kib=params.memory_cost_kib,
    argon2_parallelism=params.parallelism,
    argon2_hash_len=params.hash_len,
)
vault_path = config.profile_password_vault(profile_id)
vault_path.parent.mkdir(parents=True, exist_ok=True)
vault_path.write_text(json.dumps(asdict(rec), indent=2), encoding="utf-8")
config._restrict_permissions(vault_path)

def verify_password_and_get_master_key(profile_id: str, password: str) -> bytearray:
    """Returns that folder's master key on success. Raises
    AuthenticationError on wrong password, or CorruptedFileError if the
    vault is unreadable."""
    vault_path = config.profile_password_vault(profile_id)
    if not vault_path.exists():
        raise CorruptedFileError("No credentials found for this folder.")
    try:
        rec = json.loads(vault_path.read_text(encoding="utf-8"))
        salt = base64.b64decode(rec["salt_b64"])
        nonce = base64.b64decode(rec["nonce_b64"])
        wrapped = base64.b64decode(rec["wrapped_key_b64"])
        params = _params_from_record(rec)
    except (json.JSONDecodeError, KeyError, ValueError) as exc:
        raise CorruptedFileError("Password vault file is corrupted.") from exc

    kek = security.derive_key_argon2id(password, salt, params)
    try:
        return security.unwrap_master_key(kek, nonce, wrapped)
    finally:
        security.best_effort_zero(kek)

def change_password(profile_id: str, old_password: str, new_password: str) -> None:
    """Requires that folder's correct current password. Re-wraps the *same*
    master key under a freshly derived KEK with a new random salt/nonce, so
    the folder never needs to be re-encrypted when its password changes."""
    ok, problems = validate_password(new_password)
    if not ok:
        raise ValueError("New password does not meet policy: " + "; ".join(problems))

    master_key = verify_password_and_get_master_key(profile_id, old_password)
    try:
        create_password_vault(profile_id, new_password, master_key)
    finally:
        security.best_effort_zero(master_key)

```

## face\_auth.py

*Camera, Haar detection, LBPH enrollment/verification, liveness, encrypted templates.*

416 lines

```

"""
face_auth.py
-----
Face enrollment and verification built entirely on OpenCV (`opencv-contrib-
python`): Haar cascade classifiers for face/eye detection and the LBPH
(Local Binary Patterns Histograms) face recognizer for matching. Both are
long-established, published, reputable algorithms bundled with OpenCV
itself – not a custom/home-grown recognition algorithm.

```

Why OpenCV/LBPH instead of a deep-learning embedding model (e.g. dlib's ResNet face encoder): dlib's prebuilt Windows binaries require AVX CPU instructions. On hardware without AVX (common on low-power/older laptops), dlib fails to load at all. LBPH is a classical, CPU-only algorithm with no such requirement, at the cost of somewhat lower accuracy than a modern deep embedding model under difficult lighting/pose. This is a deliberate, disclosed trade-off for broad hardware compatibility, not an attempt to claim equivalent recognition accuracy.

Important security notes (also surfaced in the GUI and README):

```

* The face template is used ONLY as an authentication gate. It is never
  used as, or to derive, the AES encryption key (see security.py /
  password_auth.py – the key comes from Argon2id over the password).
* Face template confidentiality at rest relies on a locally stored
  "device key" (security.get_or_create_device_key), NOT on the password.
  This is a deliberate trade-off: the workflow requires face verification
  to happen *before* the password is entered, so the template cannot be
  encrypted with a password-derived key. A local attacker with full
  filesystem access to this machine could therefore access both the
  encrypted template and its key. Treat face recognition here as a
  convenience/second-channel factor, not a secret on par with the
  password.
* Liveness detection here is a coarse best-effort blink check: it watches
  for the Haar eye cascade briefly failing to detect both eyes (which
  commonly happens during a natural blink) and then detecting them again.
  This is cruder than a landmark-based eye-aspect-ratio check and is NOT
  robust against a printed photo cut-out, a video replay, or a mask. It
  raises the bar against the most trivial spoof (a single static photo)
  and nothing more.
"""

from __future__ import annotations

import json
import os
import sys
import tempfile
import time
from pathlib import Path
from typing import Callable, List, Optional, Tuple

import numpy as np

import config
import security
from security import SecurityError

try:
    import cv2
except ImportError as exc: # pragma: no cover
    cv2 = None
    _cv2_import_error = exc

class CameraError(SecurityError):
    """Webcam could not be opened or a frame could not be read."""

class FaceNotEnrolledError(SecurityError):
    """No face template has been enrolled yet."""

class DependencyMissingError(SecurityError):
    """A required third-party library (opencv-contrib-python) is not
    installed, or was installed without the contrib `cv2.face` module."""

def _base_dir() -> Path:
    """Directory to resolve bundled resource files against. When frozen by
    PyInstaller (--onefile or --onedir), sys._MEIPASS points at the
    extracted/bundled support-file directory; otherwise it's the directory
    containing this source file."""
    if getattr(sys, "frozen", False) and hasattr(sys, "_MEIPASS"):
        return Path(sys._MEIPASS)
    return Path(__file__).resolve().parent

RESOURCES_DIR = _base_dir() / "resources"
FRONTAL_FACE_CASCADE_PATH = RESOURCES_DIR / "haarcascade_frontalface_default.xml"
EYE_CASCADE_PATH = RESOURCES_DIR / "haarcascade_eye.xml"

FACE_SIZE = (200, 200) # normalized size every face sample is resized to
MIN_FACE_BOX_PX = 90 # minimum face bounding-box side, for sample quality
SAMPLE_SPACING_SECONDS = 0.8 # minimum time between accepted enrollment samples
EYE_HISTORY_WINDOW = 10 # frames of eye-visibility history kept for blink detection

def _require_dependencies() -> None:
    if cv2 is None:
        raise DependencyMissingError(
            "opencv-contrib-python is not installed. Run: pip install opencv-contrib-python"
        ) from _cv2_import_error
    if not hasattr(cv2, "face") or not hasattr(cv2.face, "LBPHFaceRecognizer_create"):
```



```

        raise DependencyMissingError(
            "The installed OpenCV build is missing the 'face' (contrib) module. "
            "Install the contrib build: pip uninstall opencv-python -y && "
            "pip install opencv-contrib-python"
        )

StatusCB = Optional[Callable[[str], None]]
FrameCB = Optional[Callable[[np.ndarray], None]]
CancelCB = Optional[Callable[[], bool]]

class FaceCamera:
    """Thin, error-checked wrapper around cv2.VideoCapture."""

    def __init__(self, camera_index: int = 0):
        _require_dependencies()
        self.camera_index = camera_index
        self.cap = None

    def _enter_(self) -> "FaceCamera":
        backend = cv2.CAP_DSHOW if sys.platform == "win32" else cv2.CAP_ANY
        self.cap = cv2.VideoCapture(self.camera_index, backend)
        if self.cap is None or not self.cap.isOpened():
            raise CameraError(
                "Could not open the webcam. Make sure a camera is connected, "
                "not already in use by another application, and that this "
                "app has camera permission."
            )
        return self

    def read_frame(self, timeout: float = 5.0):
        if self.cap is None:
            raise CameraError("Camera is not open.")
        start = time.time()
        while time.time() - start < timeout:
            ok, frame = self.cap.read()
            if ok and frame is not None:
                return frame
        raise CameraError("Timed out waiting for a frame from the webcam.")

    def _exit_(self, exc_type, exc, tb) -> None:
        if self.cap is not None:
            self.cap.release()
            self.cap = None

class _Detector:
    """Loads the bundled Haar cascades once and exposes face/eye detection
    helpers used by both enrollment and verification."""

    def __init__(self):
        _require_dependencies()
        self.face_cascade = cv2.CascadeClassifier(str(FRONTAL_FACE_CASCADE_PATH))
        if self.face_cascade.empty():
            raise DependencyMissingError(
                f"Could not load face detection data from {FRONTAL_FACE_CASCADE_PATH}."
            )
        self.eye_cascade = cv2.CascadeClassifier(str(EYE_CASCADE_PATH))
        if self.eye_cascade.empty():
            raise DependencyMissingError(
                f"Could not load eye detection data from {EYE_CASCADE_PATH}."
            )

    def detect_single_face(self, gray_frame: np.ndarray) -> Tuple[Optional[Tuple[int, int, int, int]], str]:
        """Returns ((x, y, w, h), "") on exactly one sufficiently large face,
        or (None, reason) otherwise."""
        faces = self.face_cascade.detectMultiScale(
            gray_frame, scaleFactor=1.1, minNeighbors=6, minSize=(MIN_FACE_BOX_PX, MIN_FACE_BOX_PX)
        )
        if len(faces) == 0:
            return None, "No face detected. Please look at the camera."
        if len(faces) > 1:
            return None, "Multiple faces detected. Only one person may be in frame."
        return tuple(faces[0]), ""

    def eyes_visible(self, gray_face_roi: np.ndarray) -> bool:
        eyes = self.eye_cascade.detectMultiScale(
            gray_face_roi, scaleFactor=1.1, minNeighbors=8, minSize=(18, 18)
        )
        return len(eyes) >= 2

    def _preprocess_face(gray_frame: np.ndarray, box: Tuple[int, int, int, int]) -> np.ndarray:
        x, y, w, h = box
        roi = gray_frame[y : y + h, x : x + w]

```

```

roi = cv2.resize(roi, FACE_SIZE, interpolation=cv2.INTER_AREA)
roi = cv2.equalizeHist(roi) # standard LBPH preprocessing to reduce lighting sensitivity
return roi

```

```

class FaceAuthenticator:
    """Face enrollment/verification for ONE folder's credential profile.
    Each protected folder has its own independently enrolled face, stored
    under its own profile directory – the face that opens one folder has
    no bearing on any other folder."""

    def __init__(self, profile_id: str, settings: Optional[dict] = None):
        self.profile_id = profile_id
        self.settings = settings or config.get_settings()

    @property
    def template_path(self) -> Path:
        return config.profile_face_template(self.profile_id)

    # -- template persistence -----

    def is_enrolled(self) -> bool:
        return self.template_path.exists()

    def _load_recognizer(self):
        _require_dependencies()
        if not self.is_enrolled():
            raise FaceNotEnrolledError("No face has been enrolled for this folder yet.")
        device_key = security.get_or_create_device_key(config.DEVICE_KEY_FILE)
        raw = self.template_path.read_bytes()
        if len(raw) < security.NONCE_SIZE:
            raise security.CorrruptedFileError("Face template file is corrupted.")
        nonce, ciphertext = raw[: security.NONCE_SIZE], raw[security.NONCE_SIZE :]
        model_bytes = security.aes_gcm_decrypt(
            device_key, nonce, ciphertext, aad=b"folderlock-face-template-v2"
        )

        recognizer = cv2.face.LBPHFaceRecognizer_create()
        with tempfile.TemporaryDirectory(prefix="flock_face_") as tmp:
            tmp_path = Path(tmp) / "model.xml"
            tmp_path.write_bytes(model_bytes)
            recognizer.read(str(tmp_path))
        return recognizer

    def _save_recognizer(self, recognizer) -> None:
        with tempfile.TemporaryDirectory(prefix="flock_face_") as tmp:
            tmp_path = Path(tmp) / "model.xml"
            recognizer.write(str(tmp_path))
            model_bytes = tmp_path.read_bytes()

        device_key = security.get_or_create_device_key(config.DEVICE_KEY_FILE)
        nonce = security.random_bytes(security.NONCE_SIZE)
        ciphertext = security.aes_gcm_encrypt(
            device_key, nonce, model_bytes, aad=b"folderlock-face-template-v2"
        )
        self.template_path.parent.mkdir(parents=True, exist_ok=True)
        self.template_path.write_bytes(nonce + ciphertext)
        try:
            os.chmod(self.template_path, 0o600)
        except OSError:
            pass

    # -- enrollment -----

    def enroll(
        self,
        camera_index: int = 0,
        num_samples: Optional[int] = None,
        on_status: StatusCB = None,
        on_frame: FrameCB = None,
        should_cancel: CancelCB = None,
    ) -> None:
        """Capture `num_samples` distinct, single-face frames and train a
        fresh LBPH model on them as the authorized face template (overwrites
        any existing template – callers are responsible for enforcing the
        re-enrollment authorization rule *before* calling this)."""
        _require_dependencies()
        detector = _Detector()
        num_samples = num_samples or int(self.settings["face_min_samples"])
        collected: List[np.ndarray] = []
        last_capture_time = 0.0

        def status(msg: str) -> None:
            if on_status:
                on_status(msg)

        status("Starting camera...")

```



```

        if not liveness_enabled:
            status("Face recognized")
            return True
        if blink_detected:
            status("Face recognized (liveness confirmed)")
            return True
        status("Face recognized – please blink naturally to confirm liveness...")
    else:
        status("Face not recognized")

    if matched and liveness_enabled and not blink_detected:
        status("Liveness could not be confirmed (no blink detected). Access denied.")
    else:
        status("Face not recognized. Access denied.")
    return False

def enrolled_profile_ids() -> List[str]:
    """Profile ids that currently have a face template on disk."""
    profiles = config.profiles_dir()
    if not profiles.exists():
        return []
    return [
        d.name for d in sorted(profiles.iterdir())
        if d.is_dir() and config.profile_face_template(d.name).exists()
    ]

def rotate_device_key_all_profiles() -> int:
    """Re-encrypts EVERY profile's face template under a freshly generated
    device key, and returns how many were re-encrypted.

    All templates share one device key, so this must load them all with the
    old key *before* replacing it, then write them all back with the new
    one. Rotating a single profile in isolation would silently render every
    other folder's enrolled face undecryptable.

    This only changes the at-rest storage key. It does not change any
    enrolled face and makes no authentication decision, so it does not
    require face/password verification. See the module docstring for why
    the device key cannot be password-derived.
    """
    _require_dependencies()
    ids = enrolled_profile_ids()
    loaded = [(pid, FaceAuthenticator(pid)._load_recognizer()) for pid in ids]

    if config.DEVICE_KEY_FILE.exists():
        config.DEVICE_KEY_FILE.unlink()

    for pid, recognizer in loaded:
        FaceAuthenticator(pid)._save_recognizer(recognizer)
    return len(loaded)

```

## access\_control.py

*Windows NTFS access restriction, with the reversibility safety net.*

185 lines

```

"""
access_control.py
-----
Windows NTFS access-control layer: makes a locked folder refuse to open in
Explorer (or any other program running as this user) by applying an
explicit "Deny Read & Execute" ACE for the current user, and removing that
ACE again on unlock.

WHAT THIS IS AND IS NOT
~~~~~
This is a genuine use of the operating system's own permission system –
not a trick, and not an attempt to bypass or subvert OS security. It uses
the stock `icacls` tool exactly as an administrator would.

It is, however, a *deterrent*, not an unbreakable barrier:

* The folder's owner (you) always retains the WRITE_DAC right, so you can
  always change the permission back through Explorer's Security tab or
  `icacls`. Windows Explorer will even offer an administrator a "Continue"
  button that restores access in one click.
* An administrator, or anyone who can take ownership, can remove the ACE.
* Copying the folder to a non-NTFS volume (FAT32/exFAT, many USB sticks)
  or into a ZIP silently drops ACLs entirely.

```

That is why this layer is deliberately the \*second\* line of defence. The real protection is that every file inside is AES-256-GCM ciphertext: even with access fully restored, the contents are unreadable without the password. Anyone claiming a user-space application can make a folder truly un-openable on Windows without a kernel-mode filesystem driver is overselling; this module does the strongest honest thing available.

Deliberately only "Read & Execute" is denied, never Delete. You must always be able to remove your own folder, even if this application is uninstalled or stops working.

```
from __future__ import annotations

import os
import subprocess
import sys
from pathlib import Path
from typing import Optional, Tuple

# Avoid a console window flashing up when the frozen --windowed .exe shells
# out to icacis.
_NO_WINDOW = getattr(subprocess, "CREATE_NO_WINDOW", 0) if sys.platform == "win32" else 0

# Object+Container inherit; deny FILE_LIST_DIRECTORY / FILE_READ_DATA ONLY.
#
# This exact right is load-bearing and must not be "strengthened" casually.
# Denying the broader (RX) also denies READ_CONTROL (permission to read the
# ACL) and traverse. On folders whose DACL grants the user access solely
# through an inherited OWNER RIGHTS ACE – which is what Windows produces in
# several common cases – that leaves nothing able to read the ACL in order
# to undo it, and the folder becomes permanently unreachable without an
# elevated administrator running takeown. That is unacceptable for user
# data. (RD) blocks opening the folder just as effectively while leaving
# READ_CONTROL and WRITE_DAC intact, so the restriction can always be
# lifted again. Verified against the worst-case DACL.
_DENY_RIGHTS = "(OI)(CI)(RD)"

class AccessControlError(Exception):
    """Raised when an ACL operation fails in a way the caller must know
    about (e.g. unlock could not restore access)."""

def is_available() -> bool:
    """ACL enforcement is Windows/NTFS only. Everywhere else the app still
    works, just without this extra layer (encryption is unaffected)."""
    return sys.platform == "win32"

def _run_icacis(args: list[str]) -> Tuple[int, str]:
    try:
        proc = subprocess.run(
            ["icacis", *args],
            capture_output=True,
            text=True,
            errors="replace",
            creationflags=_NO_WINDOW,
        )
        return proc.returncode, (proc.stdout or "") + (proc.stderr or "")
    except FileNotFoundError as exc:
        raise AccessControlError("Windows 'icacis' tool was not found.") from exc
    except OSError as exc:
        raise AccessControlError(f"Could not run 'icacis': {exc}") from exc

def current_user() -> str:
    """Returns DOMAIN\\User for the account this process runs as."""
    try:
        proc = subprocess.run(
            ["whoami"], capture_output=True, text=True, errors="replace",
            creationflags=_NO_WINDOW,
        )
        name = (proc.stdout or "").strip()
        if name:
            return name
    except OSError:
        pass
    domain = os.environ.get("USERDOMAIN", "")
    user = os.environ.get("USERNAME", "")
    if domain and user:
        return f"{domain}\\{user}"
    if user:
        return user
    raise AccessControlError("Could not determine the current Windows user account.")
```

```

def _can_read_acl(folder_path: Path) -> bool:
    """Whether we can still read the folder's ACL – i.e. whether we are
    still able to undo a restriction we applied."""
    return _run_icacIs([str(folder_path)])[0] == 0

def deny_access(folder_path: Path) -> None:
    """Blocks this user from opening/reading the folder.

    Applies a hard safety net: immediately after applying the restriction,
    it verifies the ACL is still readable (i.e. the restriction can still
    be lifted). If that check fails for any reason, the restriction is
    reverted at once and an error is raised. A folder must never be left
    in a state this application cannot undo.
    """
    if not is_available():
        raise AccessControlError("Folder access restriction is only supported on Windows (NTFS).")
    user = current_user()
    code, output = _run_icacIs([str(folder_path), "/deny", f"{user}:{_DENY_RIGHTS}"])
    if code != 0:
        raise AccessControlError(
            "Could not restrict folder access. This usually means the drive is "
            f"not NTFS-formatted. Details: {output.strip()[:300]}"
        )

    if not _can_read_acl(folder_path):
        _run_icacIs([str(folder_path), "/remove:d", user])
        raise AccessControlError(
            "Folder access restriction was reverted: applying it would have left "
            "the folder impossible to restore without administrator rights. The "
            "folder's contents remain encrypted."
        )

def restore_access(folder_path: Path) -> None:
    """Removes the Deny ACE this app applied, restoring normal access.
    Safe to call when no ACE is present (icacIs treats it as a no-op)."""
    if not is_available():
        return
    user = current_user()
    code, output = _run_icacIs([str(folder_path), "/remove:d", user])
    if code != 0:
        raise AccessControlError(
            "Could not restore folder access. You can restore it manually by "
            f"running: icacIs \"{folder_path}\" /remove:d \"{user}\" "
            f"Details: {output.strip()[:300]}"
        )

def is_access_denied(folder_path: Path) -> bool:
    """Best-effort check of whether the folder is currently unreadable to
    us. Used only for diagnostics/UI hints, never as the authoritative
    lock-state source (that is the locked-folder registry)."""
    try:
        os.listdir(folder_path)
        return False
    except PermissionError:
        return True
    except OSError:
        return False

def manual_restore_command(folder_path: Path) -> Optional[str]:
    """The exact command a user can run to recover access themselves if
    this application is ever unavailable. Surfaced in the UI and docs so
    nobody can be permanently locked out of their own folder by this app."""
    if not is_available():
        return None
    try:
        user = current_user()
    except AccessControlError:
        user = "%USERNAME%"
    return f'icacIs "{folder_path}" /remove:d "{user}"'

```

## encryption.py

*Streaming chunked AES-256-GCM container.*

210 lines

"""

encryption.py

-----  
Streaming, chunked, authenticated file encryption built on AES-256-GCM.

Container format (all integers big-endian / network order)

```
~~~~~
MAGIC          8 bytes    b"FLOCKv1\x00"
file_salt      16 bytes   random, used to derive this file's AES key
nonce_prefix   4 bytes    random, combined with a chunk counter
chunk_size     4 bytes    uint32, plaintext bytes per chunk
original_size  8 bytes    uint64, total plaintext size
--- repeated chunks until original_size bytes have been produced ---
chunk_len      4 bytes    uint32, length of this ciphertext chunk
chunk_ciphertext chunk_len bytes (AES-GCM ciphertext, tag included)
```

Per-file key: HKDF-SHA256(master\_key, salt=file\_salt, info=b"folderlock-file-v1")

Per-chunk nonce: nonce\_prefix (4 bytes) || chunk\_index (8 bytes, big-endian)

Per-chunk AAD: aad\_context || b"|" || chunk\_index (8 bytes BE) || b"|" || (b"LAST" or b"MORE")

Binding the caller-supplied `aad\_context` (the file's relative path within the protected folder) into every chunk's authentication tag means a chunk cannot be silently moved to a different file/path, reordered, duplicated, or truncated without detection: any such change fails AES-GCM tag verification and raises `TamperDetectedError/CorruptedFileError`.

Files are streamed in fixed-size chunks so encrypting/decrypting does not require loading an entire (potentially huge) file into memory.

```
from __future__ import annotations
```

```
import hashlib
import struct
from pathlib import Path
from typing import Callable, Optional
```

```
import config
import security
from security import CorruptedFileError, TamperDetectedError
```

```
_HEADER_STRUCT = struct.Struct(">8s16s4sIQ") # magic, salt, nonce_prefix, chunk_size, orig_size
_CHUNK_LEN_STRUCT = struct.Struct(">I")
```

```
ProgressCB = Optional[Callable[[int, int], None]] # (bytes_done, bytes_total)
```

```
def _derive_file_key(master_key: bytes, file_salt: bytes) -> bytearray:
    return security.hkdf_derive(master_key, salt=file_salt, info=b"folderlock-file-v1")
```

```
def _chunk_nonce(nonce_prefix: bytes, chunk_index: int) -> bytes:
    return nonce_prefix + chunk_index.to_bytes(8, "big")
```

```
def _chunk_aad(aad_context: bytes, chunk_index: int, is_last: bool) -> bytes:
    marker = b"LAST" if is_last else b"MORE"
    return aad_context + b"|" + chunk_index.to_bytes(8, "big") + b"|" + marker
```

```
def encrypt_file(
    src_path: Path,
    dst_path: Path,
    master_key: bytes,
    aad_context: bytes,
    progress_cb: ProgressCB = None,
) -> None:
    """Encrypt src_path -> dst_path using a fresh random salt/nonce prefix.
    dst_path's parent directory must already exist."""
    settings = config.get_settings()
    chunk_size = int(settings["chunk_size_bytes"])

    file_salt = security.random_bytes(security.SALT_SIZE)
    nonce_prefix = security.random_bytes(4)
    file_key = _derive_file_key(master_key, file_salt)

    original_size = src_path.stat().st_size
    bytes_done = 0

    tmp_dst = dst_path.with_name(dst_path.name + ".part")
    try:
        with open(src_path, "rb") as fin, open(tmp_dst, "wb") as fout:
            fout.write(_HEADER_STRUCT.pack(
                config.MAGIC, file_salt, nonce_prefix, chunk_size, original_size
            ))
```

```

        chunk_index = 0
        while True:
            plaintext = fin.read(chunk_size)
            if not plaintext and original_size > 0:
                break
            is_last = (bytes_done + len(plaintext)) >= original_size
            nonce = _chunk_nonce(nonce_prefix, chunk_index)
            aad = _chunk_aad(aad_context, chunk_index, is_last)
            ciphertext = security.aes_gcm_encrypt(file_key, nonce, plaintext, aad)
            fout.write(_CHUNK_LEN_STRUCT.pack(len(ciphertext)))
            fout.write(ciphertext)

            bytes_done += len(plaintext)
            chunk_index += 1
            if progress_cb:
                progress_cb(bytes_done, original_size)
            if is_last:
                break
        tmp_dst.replace(dst_path)
    finally:
        security.best_effort_zero(file_key)
        if tmp_dst.exists():
            try:
                tmp_dst.unlink()
            except OSError:
                pass

def decrypt_file(
    src_path: Path,
    dst_path: Path,
    master_key: bytes,
    aad_context: bytes,
    progress_cb: ProgressCB = None,
) -> None:
    """Decrypt src_path -> dst_path. Raises TamperDetectedError if any
    chunk fails authentication, or CorruptedFileError if the container is
    structurally invalid. dst_path is only ever written to a temporary
    sibling file and atomically renamed on full success, so a failed or
    interrupted decryption never leaves a partial/corrupt plaintext file
    at the final destination."""
    tmp_dst = dst_path.with_name(dst_path.name + ".part")
    try:
        with open(src_path, "rb") as fin:
            header = fin.read(_HEADER_STRUCT.size)
            if len(header) != _HEADER_STRUCT.size:
                raise CorruptedFileError("Encrypted file is truncated (missing header).")
            magic, file_salt, nonce_prefix, chunk_size, original_size = _HEADER_STRUCT.unpack(header)
            if magic != config.MAGIC:
                raise CorruptedFileError("Encrypted file has an unrecognized format.")

            file_key = _derive_file_key(master_key, file_salt)
            try:
                dst_path.parent.mkdir(parents=True, exist_ok=True)
                with open(tmp_dst, "wb") as fout:
                    bytes_done = 0
                    chunk_index = 0
                    while bytes_done < original_size:
                        len_bytes = fin.read(_CHUNK_LEN_STRUCT.size)
                        if len(len_bytes) != _CHUNK_LEN_STRUCT.size:
                            raise CorruptedFileError("Encrypted file is truncated (missing chunk length).")
                        (chunk_len,) = _CHUNK_LEN_STRUCT.unpack(len_bytes)
                        ciphertext = fin.read(chunk_len)
                        if len(ciphertext) != chunk_len:
                            raise CorruptedFileError("Encrypted file is truncated (missing chunk data).")

                        # AES-GCM appends a fixed 16-byte tag, so the plaintext
                        # length of this chunk is known before decrypting,
                        # which lets us compute `is_last` exactly (no guessing).
                        if chunk_len < 16:
                            raise CorruptedFileError("Encrypted file has an invalid chunk length.")
                        plaintext_len = chunk_len - 16
                        is_last = (bytes_done + plaintext_len) >= original_size
                        nonce = _chunk_nonce(nonce_prefix, chunk_index)
                        aad = _chunk_aad(aad_context, chunk_index, is_last)
                        plaintext = security.aes_gcm_decrypt(file_key, nonce, ciphertext, aad)

                        fout.write(plaintext)
                        bytes_done += len(plaintext)
                        chunk_index += 1
                        if progress_cb:
                            progress_cb(bytes_done, original_size)

                    if original_size == 0:
                        len_bytes = fin.read(_CHUNK_LEN_STRUCT.size)
                        if len(len_bytes) == _CHUNK_LEN_STRUCT.size:
                            (chunk_len,) = _CHUNK_LEN_STRUCT.unpack(len_bytes)
                            ciphertext = fin.read(chunk_len)
                            aad = _chunk_aad(aad_context, 0, True)
                            security.aes_gcm_decrypt(file_key, nonce_prefix + (0).to_bytes(8, "big"), ciphertext, aad)

```



```

        trailing = fin.read(1)
        if trailing:
            raise CorruptedFileError("Encrypted file has unexpected trailing data.")

        if bytes_done != original_size:
            raise CorruptedFileError("Decrypted size does not match the recorded original size.")

    finally:
        security.best_effort_zero(file_key)
        tmp_dst.replace(dst_path)
except (TamperDetectedError, CorruptedFileError):
    if tmp_dst.exists():
        try:
            tmp_dst.unlink()
        except OSError:
            pass
    raise
finally:
    if tmp_dst.exists():
        try:
            tmp_dst.unlink()
        except OSError:
            pass

def sha256_of_file(path: Path, chunk_size: int = 4 * 1024 * 1024) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            block = f.read(chunk_size)
            if not block:
                break
            h.update(block)
    return h.hexdigest()

```

## folder\_manager.py

*Recursive lock/unlock, manifest, registry, credential profiles, safe swap.*

585 lines

```

"""
folder_manager.py
-----
Recursive folder locking (encryption) and unlocking (decryption), built on
top of the streaming AEAD container format in encryption.py.

```

The folder's name and location NEVER change. Locking a folder called "TaxDocs2024" leaves it at the same path, still called "TaxDocs2024" – only its *\*contents\** change (plaintext files -> `.enc` files plus an encrypted manifest, or back again on unlock).

Two independent protections are applied when locking:

1. ENCRYPTION (the real protection). Every file's contents become AES-256-GCM ciphertext. Unreadable without the password, no matter who has access to the disk.
2. ACCESS RESTRICTION (a deterrent, Windows/NTFS only). An explicit "Deny Read" permission is applied so the folder refuses to open in Explorer at all. See `access_control.py` for an honest account of what this does and does not stop – the folder's owner can always undo it through Windows' own security dialog, which is exactly why protection #1 is the one that actually matters.

Because protection #2 makes the folder unreadable to this app too, lock state can no longer be detected by looking for the manifest *\*inside\** the folder. A registry of locked folder paths is therefore kept in the app's own vault directory, and is the authoritative source for `is_locked()` (with the in-folder manifest as a fallback for folders locked without access restriction).

### Safety model

~~~~~  
All operations for a given lock/unlock are confined to the user-selected folder and two sibling temp paths derived from its name (a hidden working directory and a hidden "previous version" holding directory). Nothing outside that is ever read, written, or deleted.

Both locking and unlocking build the new version *\*completely\** in a hidden temp sibling directory first (encrypting/decrypting every file and verifying it), while the folder at the real path is left completely untouched. Only once that new version is fully built and verified does the swap happen, and the swap itself is done as two back-to-back directory *\*renames\** (not copies – instant regardless of folder size, since they're

just filesystem directory-entry updates on the same volume), with the previous version kept under a hidden name until the very end:

1. rename <folder> -> <folder>.<op>\_old\_<uuid> (the real path briefly does not exist, but the complete original data is simply sitting under a different name right next to it – nothing was deleted, and this step is effectively instantaneous)
2. rename <temp working dir> -> <folder> (the new, already-fully-verified version is now live under the real name – also instantaneous)
3. delete <folder>.<op>\_old\_<uuid> (only now, once the new version is confirmed in place, is the old version's disk space reclaimed)

If interrupted before step 1, the folder is completely untouched and only the incomplete working temp directory needs cleaning up. If interrupted between steps 1 and 2 (an exceedingly small window – a directory rename on the same volume is not a bulk data operation), the previous version is still fully intact under its `<op>\_old\_<uuid>` name and can be renamed back manually. If interrupted between steps 2 and 3, the folder is already correctly in its new state; the leftover `<op>\_old\_<uuid>` directory is simply redundant and safe to delete.

```
from __future__ import annotations

import json
import os
import shutil
import time
import uuid
from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, List, Optional

import access_control
import config
import encryption
from access_control import AccessControlError
from security import SecurityError

ProgressCB = Optional[Callable[[str], None]]

class FolderManagerError(SecurityError):
    pass

MANIFEST_FILENAME = config.MANIFEST_RELPATH + ".enc"

@dataclass
class ManifestEntry:
    relpath: str
    sha256: str
    size: int

@dataclass
class Manifest:
    version: int
    original_name: str
    created_at: float
    files: List[ManifestEntry] = field(default_factory=list)
    empty_dirs: List[str] = field(default_factory=list)

    def to_json_bytes(self) -> bytes:
        return json.dumps(
            {
                "version": self.version,
                "original_name": self.original_name,
                "created_at": self.created_at,
                "files": [f.__dict__ for f in self.files],
                "empty_dirs": self.empty_dirs,
            },
            indent=2,
        ).encode("utf-8")

    @staticmethod
    def from_json_bytes(data: bytes) -> "Manifest":
        obj = json.loads(data.decode("utf-8"))
        return Manifest(
            version=obj["version"],
            original_name=obj["original_name"],
            created_at=obj["created_at"],
            files=[ManifestEntry(**f) for f in obj["files"]],
        )
```

```

        empty_dirs=obj.get("empty_dirs", []),
    )

# -----
# Locked-folder registry
#
# Authoritative record of which folders this app has locked. Required
# because an access-restricted folder cannot be read by this app either,
# so the in-folder manifest is not always visible (see module docstring).
# -----

def _norm(folder_path: Path) -> str:
    return str(Path(folder_path).resolve())

def _same_path(a: str, b: str) -> bool:
    # Windows paths are case-insensitive; POSIX paths are not.
    if os.name == "nt":
        return a.casefold() == b.casefold()
    return a == b

def _load_registry() -> List[dict]:
    _migrate_legacy_layout_if_needed()
    registry_file = config.folder_registry_file()
    if not registry_file.exists():
        return []
    try:
        data = json.loads(registry_file.read_text(encoding="utf-8"))
        entries = data.get("folders", [])
        return [e for e in entries if isinstance(e, dict) and "path" in e]
    except (json.JSONDecodeError, OSError):
        # A damaged registry must never make existing locked folders
        # unrecoverable – is_locked() still falls back to the in-folder
        # manifest, and unlock still works on any folder we can read.
        return []

def _save_registry(entries: List[dict]) -> None:
    registry_file = config.folder_registry_file()
    registry_file.parent.mkdir(parents=True, exist_ok=True)
    registry_file.write_text(
        json.dumps({"version": 2, "folders": entries}, indent=2), encoding="utf-8"
    )
    config._restrict_permissions(registry_file)

def _migrate_legacy_layout_if_needed() -> None:
    """Version 1 of this app used a single global password + face for every
    folder. If that layout is present, fold it into one profile and point
    the folders it locked at that profile, so they remain unlockable with
    the credentials the user already has. Runs once; never destroys the
    legacy files, only copies them forward."""
    legacy_registry = config.legacy_registry_file()
    legacy_vault = config.legacy_password_vault_file()
    if not legacy_registry.exists() and not legacy_vault.exists():
        return
    if config.folder_registry_file().exists():
        return # already migrated

    legacy_profile = "legacy-shared"
    try:
        if legacy_vault.exists():
            dst = config.profile_password_vault(legacy_profile)
            dst.parent.mkdir(parents=True, exist_ok=True)
            if not dst.exists():
                shutil.copy2(legacy_vault, dst)
        legacy_face = config.legacy_face_template_file()
        if legacy_face.exists():
            dst_face = config.profile_face_template(legacy_profile)
            dst_face.parent.mkdir(parents=True, exist_ok=True)
            if not dst_face.exists():
                shutil.copy2(legacy_face, dst_face)

        entries: List[dict] = []
        if legacy_registry.exists():
            data = json.loads(legacy_registry.read_text(encoding="utf-8"))
            for old in data.get("folders", []):
                if not isinstance(old, dict) or "path" not in old:
                    continue
                entries.append({
                    "path": old["path"],
                    "profile_id": legacy_profile,
                    "locked": True,
                    "locked_at": old.get("locked_at", time.time()),
                })
    
```

```

        "access_restricted": bool(old.get("access_restricted")),
    })
    _save_registry(entries)
except (OSError, json.JSONDecodeError):
    # Migration is best-effort. Failing it must not block the app; the
    # legacy files are left untouched for manual recovery.
    pass

def _registry_entry(folder_path: Path) -> Optional[dict]:
    target = _norm(folder_path)
    for entry in _load_registry():
        if _same_path(entry["path"], target):
            return entry
    return None

def _upsert_entry(folder_path: Path, **fields) -> None:
    target = _norm(folder_path)
    entries = _load_registry()
    for entry in entries:
        if _same_path(entry["path"], target):
            entry.update(fields)
            break
    else:
        base = {"path": target, "profile_id": None, "locked": False}
        base.update(fields)
        entries.append(base)
    _save_registry(entries)

def new_profile_id() -> str:
    """A fresh, random identifier for a folder's credential profile. Random
    rather than derived from the path, so profile directory names leak
    nothing about which folders are protected, and so a folder can be
    renamed without invalidating its credentials."""
    return uuid.uuid4().hex

def delete_profile(profile_id: str) -> None:
    """Permanently removes a credential profile's stored password vault and
    face template. Only ever used to discard a profile that was abandoned
    part-way through setup – never for a profile that is protecting a
    locked folder, since that would make the folder undecryptable."""
    if not profile_id:
        return
    in_use = any(
        e.get("profile_id") == profile_id and e.get("locked")
        for e in _load_registry()
    )
    if in_use:
        raise FolderManagerError(
            "Refusing to delete credentials that are protecting a locked folder."
        )
    shutil.rmtree(config.profile_dir(profile_id), ignore_errors=True)

def profile_id_for(folder_path: Path) -> Optional[str]:
    entry = _registry_entry(folder_path)
    return entry.get("profile_id") if entry else None

def assign_profile(folder_path: Path, profile_id: str) -> None:
    """Associates a credential profile with a folder without locking it."""
    _upsert_entry(folder_path, profile_id=profile_id)

def has_credentials(folder_path: Path) -> bool:
    """Whether this folder already has its own password + face enrolled."""
    profile_id = profile_id_for(folder_path)
    if not profile_id:
        return False
    return config.profile_password_vault(profile_id).exists()

def list_known_folders() -> List[dict]:
    """Every folder this app has credentials for, locked or not, newest
    first. 'exists' reports whether the folder is still on disk."""
    entries = sorted(_load_registry(), key=lambda e: e.get("locked_at", 0), reverse=True)
    for entry in entries:
        entry["exists"] = Path(entry["path"]).is_dir()
    return entries

def list_locked_folders() -> List[dict]:

```

```

"""Only the folders currently locked, newest first."""
return [e for e in list_known_folders() if e.get("locked")]

def forget_folder(folder_path: Path, delete_profile: bool = False) -> None:
    """Drops a registry entry (e.g. the folder was deleted or moved outside
    the app). Never touches the folder itself. Optionally also deletes the
    stored credentials for it, which is irreversible."""
    target = _norm(folder_path)
    entries = _load_registry()
    remaining = []
    removed_profile = None
    for entry in entries:
        if _same_path(entry["path"], target):
            removed_profile = entry.get("profile_id")
        else:
            remaining.append(entry)
    _save_registry(remaining)

    if delete_profile and removed_profile:
        still_used = any(e.get("profile_id") == removed_profile for e in remaining)
        if not still_used:
            shutil.rmtree(config.profile_dir(removed_profile), ignore_errors=True)

# Kept under the old name so existing callers/tests keep working.
forget_locked_folder = forget_folder

def is_locked(folder_path: Path) -> bool:
    """Registry is authoritative; the in-folder manifest is a fallback for
    folders locked without access restriction (or on non-Windows)."""
    entry = _registry_entry(folder_path)
    if entry is not None:
        return bool(entry.get("locked"))
    try:
        return (Path(folder_path) / MANIFEST_FILENAME).is_file()
    except OSError:
        return False

def _validate_folder(folder_path: Path) -> Path:
    folder_path = Path(folder_path).resolve()
    if not folder_path.exists() or not folder_path.is_dir():
        raise FolderManagerError("The selected path is not an existing folder.")
    if folder_path == config.APP_DATA_DIR or config.APP_DATA_DIR in folder_path.parents:
        raise FolderManagerError("Refusing to operate on the application's own data directory.")
    return folder_path

def _iter_files_and_empty_dirs(root: Path):
    files = []
    empty_dirs = []
    for dirpath, dirnames, filenames in _walk_sorted(root):
        rel_dir = Path(dirpath).relative_to(root)
        if not dirnames and not filenames and str(rel_dir) != ".":
            empty_dirs.append(rel_dir.as_posix())
        for name in filenames:
            full = Path(dirpath) / name
            rel = full.relative_to(root)
            files.append((full, rel.as_posix()))
    return files, empty_dirs

def _walk_sorted(root: Path):
    import os
    for dirpath, dirnames, filenames in os.walk(root):
        dirnames.sort()
        filenames.sort()
        yield dirpath, dirnames, filenames

def _swap_in(folder_path: Path, new_dir: Path, op: str, status) -> None:
    """Atomically-as-possible replaces folder_path's contents with new_dir's
    contents, keeping folder_path's name unchanged throughout. See module
    docstring for the safety reasoning behind this exact ordering."""
    old_dir = folder_path.parent / f".{folder_path.name}.{op}_old_{uuid.uuid4().hex[:8]}"
    status("Finalizing...")
    folder_path.rename(old_dir)
    try:
        new_dir.rename(folder_path)
    except Exception:
        # Put the original back exactly as it was before giving up.
        old_dir.rename(folder_path)
        raise
    status("Cleaning up previous version...")

```

```

shutil.rmtree(old_dir)

def lock_folder(
    folder_path: Path,
    master_key: bytes,
    profile_id: str,
    progress_cb: ProgressCB = None,
) -> Path:
    def status(msg: str) -> None:
        if progress_cb:
            progress_cb(msg)

    folder_path = _validate_folder(folder_path)
    if is_locked(folder_path):
        raise FolderManagerError("This folder is already locked.")

    temp_dir = folder_path.parent / f"{folder_path.name}.locking_tmp_{uuid.uuid4().hex[:8]}"
    temp_dir.mkdir(parents=False, exist_ok=False)

    try:
        status("Scanning folder...")
        files, empty_dirs = _iter_files_and_empty_dirs(folder_path)
        manifest = Manifest(version=1, original_name=folder_path.name, created_at=time.time())

        for index, (src_file, relpath) in enumerate(files, start=1):
            status(f"Encrypting {relpath} ({index}/{len(files)})...")
            original_sha256 = encryption.sha256_of_file(src_file)
            dst_file = temp_dir / (relpath + ".enc")
            dst_file.parent.mkdir(parents=True, exist_ok=True)
            aad = relpath.encode("utf-8")

            encryption.encrypt_file(src_file, dst_file, master_key, aad)

            # Verify before trusting this file: decrypt to a scratch file and
            # hash-compare against the original plaintext.
            scratch = dst_file.with_suffix(dst_file.suffix + ".verify")
            try:
                encryption.decrypt_file(dst_file, scratch, master_key, aad)
                if encryption.sha256_of_file(scratch) != original_sha256:
                    raise FolderManagerError(f"Verification mismatch after encrypting '{relpath}'.")
            finally:
                if scratch.exists():
                    scratch.unlink()

            manifest.files.append(
                ManifestEntry(relpath=relpath, sha256=original_sha256, size=src_file.stat().st_size)
            )

        manifest.empty_dirs = empty_dirs

        status("Writing manifest...")
        manifest_plain_tmp = temp_dir / "__manifest_plain__.tmp"
        manifest_plain_tmp.write_bytes(manifest.to_json_bytes())
        try:
            manifest_dst = temp_dir / MANIFEST_FILENAME
            encryption.encrypt_file(
                manifest_plain_tmp, manifest_dst, master_key, config.MANIFEST_RELPATH.encode("utf-8")
            )
        finally:
            manifest_plain_tmp.unlink(missing_ok=True)

        _swap_in(folder_path, temp_dir, "locking", status)

        # Encryption has fully succeeded at this point. Everything below is
        # the second, best-effort layer: if restricting access fails, the
        # folder is still properly encrypted, so we record the outcome and
        # let the caller warn rather than failing the whole operation and
        # leaving the user unsure whether their data was protected.
        access_restricted = False
        settings = config.get_settings()
        if settings.get("restrict_folder_access", True) and access_control.is_available():
            status("Restricting folder access...")
            try:
                access_control.deny_access(folder_path)
                access_restricted = True
            except AccessControlError as exc:
                status(f"Warning: folder is encrypted, but access could not be restricted. {exc}")

        _upsert_entry(
            folder_path,
            profile_id=profile_id,
            locked=True,
            locked_at=time.time(),
            access_restricted=access_restricted,
        )

```

```

        status("Encryption completed.")
        return folder_path

    except Exception:
        status("Encryption failed.")
        if temp_dir.exists():
            shutil.rmtree(temp_dir, ignore_errors=True)
        raise

def unlock_folder(
    folder_path: Path,
    master_key: bytes,
    progress_cb: ProgressCB = None,
) -> Path:
    def status(msg: str) -> None:
        if progress_cb:
            progress_cb(msg)

    folder_path = _validate_folder(folder_path)
    if not is_locked(folder_path):
        raise FolderManagerError("This folder is not locked.")

    # Access must be restored before anything can be read out of the folder
    # - including by this app. If this fails we stop immediately and tell the
    # user the exact command to fix it by hand, rather than proceeding into
    # a confusing cascade of permission errors.
    entry = _registry_entry(folder_path)
    was_restricted = bool(entry and entry.get("access_restricted")) or access_control.is_access_denied(folder_path)
    if was_restricted:
        status("Restoring folder access...")
        try:
            access_control.restore_access(folder_path)
        except AccessControlError as exc:
            raise FolderManagerError(str(exc)) from exc

    def _reapply_restriction_after_failure() -> None:
        """Leave the folder locked-and-restricted exactly as we found it if
        the unlock did not complete - a failed unlock attempt must never
        quietly downgrade the folder's protection."""
        if not was_restricted:
            return
        try:
            access_control.deny_access(folder_path)
        except AccessControlError:
            pass # already reported via the primary exception

    temp_dir = folder_path.parent / f".{folder_path.name}.unlocking_tmp_{uuid.uuid4().hex[:8]}"
    temp_dir.mkdir(parents=False, exist_ok=False)

    try:
        status("Reading manifest...")
        manifest_src = folder_path / MANIFEST_FILENAME
        manifest_plain_tmp = temp_dir / "__manifest_plain__.tmp"
        encryption.decrypt_file(
            manifest_src, manifest_plain_tmp, master_key, config.MANIFEST_RELPATH.encode("utf-8")
        )
        manifest = Manifest.from_json_bytes(manifest_plain_tmp.read_bytes())
        manifest_plain_tmp.unlink(missing_ok=True)

        total = len(manifest.files)
        for index, entry in enumerate(manifest.files, start=1):
            status(f"Decrypting {entry.relpath} ({index}/{total})...")
            src_file = folder_path / (entry.relpath + ".enc")
            if not src_file.exists():
                raise FolderManagerError(f"Encrypted file for '{entry.relpath}' is missing.")
            dst_file = temp_dir / entry.relpath
            dst_file.parent.mkdir(parents=True, exist_ok=True)
            aad = entry.relpath.encode("utf-8")

            encryption.decrypt_file(src_file, dst_file, master_key, aad)

            actual_sha256 = encryption.sha256_of_file(dst_file)
            if actual_sha256 != entry.sha256:
                raise FolderManagerError(
                    f"Integrity check failed for '{entry.relpath}': file does not match the original."
                )

        for rel_dir in manifest.empty_dirs:
            (temp_dir / rel_dir).mkdir(parents=True, exist_ok=True)

        _swap_in(folder_path, temp_dir, "unlocking", status)
        # Keep the folder's profile on record so re-locking it later reuses
        # the same password and face rather than silently starting over.
        _upsert_entry(folder_path, locked=False, access_restricted=False)

```

```

        status("Folder unlocked.")
        return folder_path

    except Exception:
        status("Decryption failed. The locked folder has not been modified.")
        if temp_dir.exists():
            shutil.rmtree(temp_dir, ignore_errors=True)
            _reapply_restriction_after_failure()
            raise

def find_stale_temp_dirs(parent: Path) -> List[Path]:
    """Locates leftover working/previous-version directories from a
    previous interrupted operation, for the user to review and remove. They
    are never deleted automatically."""
    parent = Path(parent)
    if not parent.exists():
        return []
    markers = ("locking_tmp_", "unlocking_tmp_", "locking_old_", "unlocking_old_")
    stale = []
    for child in parent.iterdir():
        if child.is_dir() and any(marker in child.name for marker in markers):
            stale.append(child)
    return stale

```

## ui.py

*All screens; enforces face-then-password structurally.*

1210 lines

```

"""
ui.py
-----
Tkinter desktop GUI. Wires together face_auth, password_auth, and
folder_manager and enforces the mandatory authentication order:

    FACE RECOGNITION --> PASSWORD --> UNLOCK / LOCK / CHANGE CREDENTIALS

The password entry screen is only ever constructed/shown *after* a
background face-verification task has reported success (see
`AuthDialog.on_face_done`). There is no code path that reveals the
password field first.
"""

from __future__ import annotations

import logging
import queue
import threading
import tkinter as tk
from pathlib import Path
from tkinter import filedialog, messagebox, ttk
from typing import Optional

import access_control
import config
import face_auth
import folder_manager
import password_auth
import security

try:
    from PIL import Image, ImageTk
except ImportError:
    Image = None
    ImageTk = None

logger = logging.getLogger("folder_lock")

MAX_PASSWORD_ATTEMPTS = 5
WINDOW_BG = "#1e1e2e"

class Status:
    FACE_NOT_RECOGNIZED = "Face not recognized"
    FACE_RECOGNIZED = "Face recognized"
    ENTER_PASSWORD = "Enter password"
    INCORRECT_PASSWORD = "Incorrect password"
    FOLDER_UNLOCKED = "Folder unlocked"
    ENCRYPTION_COMPLETED = "Encryption completed"

```



```

ENCRYPTION_FAILED = "Encryption failed"
DECRYPTION_FAILED = "Decryption failed"
ACCESS_DENIED = "ACCESS DENIED"

def _safe_error_text(exc: Exception) -> str:
    """Show a clear but non-sensitive message. Our own SecurityError
    subclasses already carry safe, user-facing text; anything unexpected is
    reported by type name only, never a raw traceback (which could contain
    file-system detail we'd rather not surface, though never secrets)."""
    if isinstance(exc, security.SecurityError):
        return str(exc)
    return f"{type(exc).__name__}: an unexpected error occurred."

PREVIEW_SIZE = (480, 360) # visible camera preview size, in pixels

STATUS_COLOR_SUCCESS = "#1b8a3a"
STATUS_COLOR_DENIED = "#c62828"
STATUS_COLOR_WARNING = "#b8860b"
STATUS_COLOR_NEUTRAL = "#1e1e1e"

def _status_color(text: str) -> str:
    """Maps a status message to a color so the user can tell at a glance
    what's happening (success / denied / needs-attention / neutral),
    without having to read closely."""
    lower = text.lower()
    if any(p in lower for p in ("not recognized", "access denied", "could not be confirmed", "cancelled", "failed")):
        return STATUS_COLOR_DENIED
    if any(p in lower for p in ("recognized", "enrolled successfully", "liveness confirmed")):
        return STATUS_COLOR_SUCCESS
    if any(p in lower for p in ("no face detected", "multiple faces", "move closer")):
        return STATUS_COLOR_WARNING
    return STATUS_COLOR_NEUTRAL

def _frame_to_photo(frame_bgr, size=PREVIEW_SIZE):
    if Image is None or ImageTk is None or frame_bgr is None:
        return None
    rgb = frame_bgr[:, :, ::-1]
    img = Image.fromarray(rgb).resize(size)
    return ImageTk.PhotoImage(img)

# -----
# Reusable widgets
# -----

class CameraPanel(tk.Frame):
    """Displays a live camera preview plus a prominent, color-coded status
    line. IMPORTANT: tk.Label's width/height are measured in *characters* if
    the label has no image, but silently switch to *pixels* the moment an
    image is set – so the preview must be given a real (blank) image from
    the start to guarantee a stable, correctly-sized pixel box throughout,
    rather than collapsing to a near-invisible size the first time a camera
    frame arrives."""

    def __init__(self, master, **kw):
        super().__init__(master, **kw)
        if Image is not None and ImageTk is not None:
            self._blank = ImageTk.PhotoImage(Image.new("RGB", PREVIEW_SIZE, (0, 0, 0)))
            self.image_label = tk.Label(self, image=self._blank, bg="black")
        else:
            self._blank = None
            self.image_label = tk.Label(self, bg="black", width=60, height=22)
        self.image_label.pack()
        self.status_label = tk.Label(
            self, text="", font=("Segoe UI", 13, "bold"),
            wraplength=PREVIEW_SIZE[0], justify="center", fg=STATUS_COLOR_NEUTRAL,
        )
        self.status_label.pack(pady=(10, 0))
        self._photo = None

    def set_status(self, text: str) -> None:
        self.status_label.config(text=text, fg=_status_color(text))

    def set_frame(self, frame_bgr) -> None:
        photo = _frame_to_photo(frame_bgr)
        if photo is not None:
            self._photo = photo # keep a reference alive
            self.image_label.config(image=photo)

```

```

class PasswordChecklist(tk.Frame):
    """Live-updating checklist of the four password requirements."""

    LABELS = [
        "6+ characters",
        "Contains a letter",
        "Contains a number",
        "Contains a symbol",
    ]

    def __init__(self, master, **kw):
        super().__init__(master, **kw)
        self.rows = []
        for text in self.LABELS:
            row = tk.Label(self, text=f"X {text}", fg="#b00020", anchor="w", justify="left")
            row.pack(fill="x")
            self.rows.append(row)

    def update_from(self, password: str) -> bool:
        settings = config.get_settings()
        min_len = settings["min_password_length"]
        checks = [
            len(password or "") >= min_len,
            any(c.isalpha() for c in (password or "")),
            any(c.isdigit() for c in (password or "")),
            any(not c.isalnum() for c in (password or "")),
        ]
        for row, ok, label in zip(self.rows, checks, self.LABELS):
            row.config(
                text=f"{'✓' if ok else 'X'} {label}",
                fg="#2e7d32" if ok else "#b00020",
            )
        return all(checks)

```

```

class BackgroundFaceTask:
    """Runs a face_auth call (enroll or verify) in a daemon thread and
    forwards status/frame/result events through a thread-safe queue for a
    Tk widget to poll on the main thread (Tk/Tcl is not thread-safe)."""

    def __init__(self, fn, **kwargs):
        self._queue: "queue.Queue" = queue.Queue()
        self._cancel_flag = threading.Event()
        self._thread = threading.Thread(target=self._run, args=(fn, kwargs), daemon=True)

    def start(self) -> None:
        self._thread.start()

    def cancel(self) -> None:
        self._cancel_flag.set()

    def _should_cancel(self) -> bool:
        return self._cancel_flag.is_set()

    def _run(self, fn, kwargs) -> None:
        try:
            result = fn(
                on_status=lambda m: self._queue.put(("status", m)),
                on_frame=lambda f: self._queue.put(("frame", f)),
                should_cancel=self._should_cancel,
                **kwargs,
            )
            self._queue.put(("done", result))
        except Exception as exc: # surfaced to the UI, never swallowed
            self._queue.put(("error", exc))

    def poll(self):
        events = []
        try:
            while True:
                events.append(self._queue.get_nowait())
        except queue.Empty:
            pass
        return events

```

```

# -----
# Mandatory FACE -> PASSWORD authentication dialog
# -----

```

```

class AuthDialog(tk.Toplevel):
    """mode="full": on success self.result = master_key (bytes).
    mode="face_only": on success self.result = True.
    Denial or cancellation always leaves self.result = None.

```

The password widgets are created but never packed/shown until the face stage reports a match – enforcing FACE -> PASSWORD at the UI level, not just by convention.

```
def __init__(self, master, profile_id: str, mode: str = "full", title: str = "Authenticate"):
    super().__init__(master)
    self.title(title)
    self.resizable(False, False)
    self.configure(padx=4, pady=4)
    self.profile_id = profile_id
    self.mode = mode
    self.result = None
    self._task: Optional[BackgroundFaceTask] = None
    self._attempts = 0

    self.transient(master)
    self.grab_set()
    self.protocol("WM_DELETE_WINDOW", self._on_cancel)

    self.camera_panel = CameraPanel(self, padx=16, pady=16)
    self.camera_panel.pack()

    self.password_frame = tk.Frame(self, padx=16, pady=8)
    tk.Label(self.password_frame, text=Status.ENTER_PASSWORD, font=("Segoe UI", 11, "bold")).pack(anchor="w")
    self.password_var = tk.StringVar()
    self.password_entry = tk.Entry(self.password_frame, textvariable=self.password_var, show="*", width=32)
    self.password_entry.pack(fill="x", pady=(4, 0))
    self.password_entry.bind("<Return>", lambda e: self._submit_password())
    self.password_status = tk.Label(self.password_frame, text="", fg="#b00020")
    self.password_status.pack(anchor="w")
    tk.Button(self.password_frame, text="Submit", command=self._submit_password).pack(anchor="e", pady=(6, 0))

    btn_frame = tk.Frame(self, padx=16, pady=8)
    btn_frame.pack(fill="x")
    self.retry_btn = tk.Button(btn_frame, text="Try Again", command=self._start_face_stage)
    tk.Button(btn_frame, text="Cancel", command=self._on_cancel).pack(side="right")

    self._start_face_stage()

def _start_face_stage(self) -> None:
    self.retry_btn.pack_forget()
    self.password_frame.pack_forget()
    self.password_var.set("")
    self.password_status.config(text="")
    self.camera_panel.set_status("Starting camera...")
    fa = face_auth.FaceAuthenticator(self.profile_id)
    self._task = BackgroundFaceTask(fa.verify)
    self._task.start()
    self.after(60, self._poll_face)

def _poll_face(self) -> None:
    if not self.wininfo_exists():
        return # dialog was closed/cancelled; stop touching destroyed widgets
    for kind, payload in self._task.poll():
        if kind == "status":
            self.camera_panel.set_status(payload)
        elif kind == "frame":
            self.camera_panel.set_frame(payload)
        elif kind == "done":
            self._on_face_done(bool(payload))
            return
        elif kind == "error":
            self._on_face_error(payload)
            return
    self.after(60, self._poll_face)

def _on_face_error(self, exc: Exception) -> None:
    logger.warning("face verification error: %s", type(exc).__name__)
    messagebox.showerror("Camera error", _safe_error_text(exc), parent=self)
    self.result = None
    self.destroy()

def _on_face_done(self, matched: bool) -> None:
    if not matched:
        logger.info("face verification denied")
        self.camera_panel.set_status(f"{Status.FACE_NOT_RECOGNIZED}\n{Status.ACCESS_DENIED}")
        self.retry_btn.pack(side="left")
        return
    logger.info("face verification succeeded")
    self.camera_panel.set_status(Status.FACE_RECOGNIZED)
    if self.mode == "face_only":
        self.result = True
        self.after(400, self.destroy)
        return
    self.after(300, self._show_password_stage)
```

```

def _show_password_stage(self) -> None:
    self.retry_btn.pack_forget()
    self.password_frame.pack()
    self.password_entry.focus_set()

def _submit_password(self) -> None:
    password = self.password_var.get()
    try:
        master_key = password_auth.verify_password_and_get_master_key(self.profile_id, password)
    except security.AuthenticationError:
        logger.info("password verification failed")
        self._attempts += 1
        remaining = MAX_PASSWORD_ATTEMPTS - self._attempts
        self.password_var.set("")
        if remaining <= 0:
            self.password_status.config(text=f"{Status.INCORRECT_PASSWORD}. {Status.ACCESS_DENIED}")
            self.result = None
            self.after(900, self.destroy)
        else:
            self.password_status.config(text=f"{Status.INCORRECT_PASSWORD} ({remaining} attempts left)")
            return
    except security.SecurityError as exc:
        messagebox.showerror("Error", _safe_error_text(exc), parent=self)
        self.result = None
        self.destroy()
        return

    logger.info("password verification succeeded")
    self.password_var.set("")
    self.result = master_key
    self.destroy()

def _on_cancel(self) -> None:
    if self._task:
        self._task.cancel()
    self.result = None
    self.destroy()

```

```

# -----
# First-time setup wizard (and face re-enrollment reuse)
# -----

```

```

class EnrollmentDialog(tk.Toplevel):
    """Standalone camera-driven enrollment screen. Used by the setup wizard
    and by the Enroll/Re-enroll Face menu action (after prior verification
    for re-enrollment, enforced by the caller)."""

    def __init__(self, master, profile_id: str, title: str = "Face Enrollment"):
        super().__init__(master)
        self.title(title)
        self.resizable(False, False)
        self.transient(master)
        self.grab_set()
        self.profile_id = profile_id
        self.success = False
        self._task: Optional[BackgroundFaceTask] = None

        tk.Label(
            self,
            text="Look directly at the camera. We will capture several samples.\n"
            "Please only one face in frame.",
            padx=16, justify="center",
        ).pack(pady=(16, 0))

        self.camera_panel = CameraPanel(self, padx=16, pady=16)
        self.camera_panel.pack()

        self.start_btn = tk.Button(self, text="Begin Enrollment", command=self._start)
        self.start_btn.pack(pady=(0, 12))
        self.protocol("WM_DELETE_WINDOW", self._on_cancel)

    def _start(self) -> None:
        self.start_btn.config(state="disabled")
        fa = face_auth.FaceAuthenticator(self.profile_id)
        self._task = BackgroundFaceTask(fa.enroll)
        self._task.start()
        self.after(60, self._poll)

    def _poll(self) -> None:
        if not self.winfo_exists():
            return # dialog was closed/cancelled; stop touching destroyed widgets
        for kind, payload in self._task.poll():
            if kind == "status":
                self.camera_panel.set_status(payload)

```

```

        elif kind == "frame":
            self.camera_panel.set_frame(payload)
        elif kind == "done":
            self.success = True
            self.after(600, self.destroy)
            return
        elif kind == "error":
            messagebox.showerror("Enrollment failed", _safe_error_text(payload), parent=self)
            self.destroy()
            return
        self.after(60, self._poll)

    def _on_cancel(self) -> None:
        if self._task:
            self._task.cancel()
            self.destroy()

```

```

class FolderCredentialsWizard(tk.Toplevel):
    """Creates the password + enrolled face for ONE folder.

```

Each folder gets its own independent credentials, so this runs once per folder rather than once per installation. On success `success` is True and `master\_key` holds that folder's freshly generated encryption key, handed straight to the caller so the folder can be locked immediately without making the user authenticate again seconds after choosing the password."""

```

    def __init__(self, master, profile_id: str, folder_path: Path, on_complete=None):
        super().__init__(master)
        self.title(f"Set Up Protection - {Path(folder_path).name}")
        self.resizable(False, False)
        self.transient(master)
        self.grab_set()
        self.protocol("WM_DELETE_WINDOW", self._on_close)
        self.profile_id = profile_id
        self.folder_path = Path(folder_path)
        self.success = False
        self.master_key: Optional[bytearray] = None
        self._on_complete = on_complete
        self._password = None
        self._active_task: Optional[BackgroundFaceTask] = None

        self.container = tk.Frame(self, padx=20, pady=20)
        self.container.pack()

        self._show_password_step()

    def _clear(self) -> None:
        for child in self.container.winfo_children():
            child.destroy()

    def _show_password_step(self) -> None:
        self._clear()
        tk.Label(self.container, text="Create Password", font=("Segoe UI", 13, "bold")).pack(anchor="w")
        tk.Label(
            self.container,
            text="These credentials protect only this folder:\n" + str(self.folder_path),
            fg="#555555", justify="left", wraplength=340,
        ).pack(anchor="w", pady=(4, 0))

        tk.Label(self.container, text="Password:").pack(anchor="w", pady=(10, 0))
        pw_var = tk.StringVar()
        pw_entry = tk.Entry(self.container, textvariable=pw_var, show="*", width=34)
        pw_entry.pack()

        tk.Label(self.container, text="Confirm password:").pack(anchor="w", pady=(8, 0))
        pw2_var = tk.StringVar()
        pw2_entry = tk.Entry(self.container, textvariable=pw2_var, show="*", width=34)
        pw2_entry.pack()

        checklist = PasswordChecklist(self.container)
        checklist.pack(fill="x", pady=(10, 0))

        match_label = tk.Label(self.container, text="", fg="#b00020")
        match_label.pack(anchor="w")

        next_btn = tk.Button(self.container, text="Next", state="disabled")
        next_btn.pack(anchor="e", pady=(12, 0))

    def on_change(* args) -> None:
        valid = checklist.update_from(pw_var.get())
        match = pw_var.get() == pw2_var.get() and pw_var.get() != ""
        match_label.config(text="" if match else "Passwords do not match.")
        next_btn.config(state="normal" if (valid and match) else "disabled")

```

```

pw_var.trace_add("write", on_change)
pw2_var.trace_add("write", on_change)

def go_next() -> None:
    ok, problems = password_auth.validate_password(pw_var.get())
    if not ok:
        messagebox.showerror("Invalid password", "\n".join(problems), parent=self)
        return
    self._password = pw_var.get()
    pw_var.set("")
    pw2_var.set("")
    self._commit_password_and_continue()

next_btn.config(command=go_next)
pw_entry.focus_set()

def _commit_password_and_continue(self) -> None:
    # This folder's own randomly generated encryption key. Deliberately
    # NOT zeroed here: the caller locks the folder with it immediately
    # after the wizard finishes, so wiping it now would force the user
    # to re-authenticate seconds after choosing their password. The
    # caller zeroes it once the lock completes.
    master_key = security.generate_master_key()
    try:
        password_auth.create_password_vault(self.profile_id, self._password, master_key)
    except Exception as exc:
        security.best_effort_zero(master_key)
        messagebox.showerror("Error", _safe_error_text(exc), parent=self)
        return
    finally:
        self._password = None
        self.master_key = master_key
        messagebox.showinfo(
            "Password accepted",
            "Password accepted for this folder.\n\nNow enroll the face that will unlock it.",
            parent=self,
        )
    self._show_face_step()

def _show_face_step(self) -> None:
    self._clear()
    tk.Label(self.container, text="Face Enrollment", font=("Segoe UI", 13, "bold")).pack(anchor="w")
    tk.Label(
        self.container,
        text="Starting camera...\nPlease look at the camera.",
        justify="center",
    ).pack(pady=(6, 6))

    cam = CameraPanel(self.container)
    cam.pack()
    start_btn = tk.Button(self.container, text="Begin Enrollment")
    start_btn.pack(pady=10)

def start() -> None:
    start_btn.config(state="disabled")
    fa = face_auth.FaceAuthenticator(self.profile_id)
    task = BackgroundFaceTask(fa.enroll)
    self._active_task = task
    task.start()

def poll() -> None:
    if not self.wininfo_exists():
        return # window was closed/cancelled; stop touching destroyed widgets
    for kind, payload in task.poll():
        if kind == "status":
            cam.set_status(payload)
        elif kind == "frame":
            cam.set_frame(payload)
        elif kind == "done":
            self._active_task = None
            self._success = True
            messagebox.showinfo(
                "Setup complete",
                f"Face enrolled successfully.\n\nThis face and "
                f"password now protect '{self.folder_path.name}' only.",
                parent=self,
            )
            if self._on_complete:
                self._on_complete()
            self.destroy()
            return
        elif kind == "error":
            self._active_task = None
            messagebox.showerror("Enrollment failed", _safe_error_text(payload), parent=self)
            start_btn.config(state="normal")
            return
    self.after(60, poll)

self.after(60, poll)

```

```

start_btn.config(command=start)

def _on_close(self) -> None:
    if not messagebox.askyesno(
        "Cancel setup?",
        "Setup for this folder is not complete, so it will NOT be locked. Cancel?",
        parent=self,
    ):
        return
    if self._active_task is not None:
        self._active_task.cancel()
    # Abandoning half-way would otherwise leave a profile holding a
    # password but no enrolled face - unusable, and it would block a
    # later attempt to set this folder up properly. Discard it.
    if not self.success:
        if self.master_key is not None:
            security.best_effort_zero(self.master_key)
            self.master_key = None
        folder_manager.delete_profile(self.profile_id)
    self.destroy()

# -----
# Security settings
# -----

class SecuritySettingsDialog(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Security Settings")
        self.resizable(False, False)
        self.transient(master)
        self.grab_set()

        settings = config.get_settings()
        frm = tk.Frame(self, padx=16, pady=16)
        frm.pack()

        row = 0
        tk.Label(frm, text="Face match threshold (LBPH confidence, lower = stricter):").grid(row=row, column=0,
        sticky="w")
        self.threshold_var = tk.DoubleVar(value=settings["face_match_threshold"])
        tk.Spinbox(frm, from_=20, to=150, increment=1, textvariable=self.threshold_var, width=8).grid(row=row, column=1)
        row += 1

        tk.Label(frm, text="Face verification timeout (seconds):").grid(row=row, column=0, sticky="w")
        self.timeout_var = tk.IntVar(value=settings["face_verify_timeout_seconds"])
        tk.Spinbox(frm, from_=10, to=90, textvariable=self.timeout_var, width=8).grid(row=row, column=1)
        row += 1

        self.liveness_var = tk.BooleanVar(value=settings["liveness_check_enabled"])
        tk.Checkbutton(frm, text="Enable blink-based liveness check", variable=self.liveness_var).grid(
            row=row, column=0, columnspan=2, sticky="w"
        )
        row += 1

        tk.Label(
            frm,
            text=(
                "Liveness check is a coarse, best-effort deterrent against a static\n"
                "photo – it watches for the eye detector briefly losing and then\n"
                "regaining both eyes, as happens during a natural blink. It is NOT a\n"
                "robust anti-spoofing measure and will not reliably stop a video\n"
                "replay attack or a printed photo held very steadily."
            ),
            fg="#8a6d00", justify="left", wraplength=380,
        ).grid(row=row, column=0, columnspan=2, sticky="w", pady=(4, 8))
        row += 1

        self.restrict_var = tk.BooleanVar(value=settings.get("restrict_folder_access", True))
        tk.Checkbutton(
            frm, text="Block locked folders from opening (Windows/NTFS only)",
            variable=self.restrict_var,
        ).grid(row=row, column=0, columnspan=2, sticky="w", pady=(6, 0))
        row += 1

        tk.Label(
            frm,
            text=(
                "Applies a Windows 'Deny Read' permission so a locked folder refuses\n"
                "to open in Explorer. This is a deterrent layered on top of encryption,\n"
                "not a replacement for it: as the folder's owner you can always restore\n"
                "access through Windows' own Security tab, and an administrator can\n"
                "always override it. Your files stay unreadable either way, because\n"
                "their contents are encrypted."
            )

```

```

    ),
    fg="#8a6d00", justify="left", wraplength=420,
).grid(row=row, column=0, columnspan=2, sticky="w", pady=(4, 8))
row += 1

tk.Label(frm, text="Argon2id time cost (applies at next password change:)").grid(row=row, column=0, sticky="w")
self.time_cost_var = tk.IntVar(value=settings["argon2_time_cost"])
tk.Spinbox(frm, from_=1, to=10, textvariable=self.time_cost_var, width=8).grid(row=row, column=1)
row += 1

tk.Label(frm, text="Argon2id memory cost (KiB):").grid(row=row, column=0, sticky="w")
self.mem_cost_var = tk.IntVar(value=settings["argon2_memory_cost_kib"])
tk.Spinbox(frm, from_=8192, to=262144, increment=8192, textvariable=self.mem_cost_var, width=8).grid(row=row,
column=1)
row += 1

tk.Label(frm, text="Argon2id parallelism:").grid(row=row, column=0, sticky="w")
self.parallelism_var = tk.IntVar(value=settings["argon2_parallelism"])
tk.Spinbox(frm, from_=1, to=8, textvariable=self.parallelism_var, width=8).grid(row=row, column=1)
row += 1

tk.Label(frm, text="App data directory:").grid(row=row, column=0, sticky="w", pady=(10, 0))
data_dir_entry = tk.Entry(frm, width=42)
data_dir_entry.grid(row=row, column=1, pady=(10, 0))
data_dir_entry.insert(0, str(config.APP_DATA_DIR))
data_dir_entry.config(state="readonly")
row += 1

tk.Button(frm, text="Rotate device key (re-encrypt face template)", command=self._rotate_device_key).grid(
    row=row, column=0, columnspan=2, sticky="we", pady=(10, 0)
)
row += 1

btns = tk.Frame(frm)
btns.grid(row=row, column=0, columnspan=2, pady=(14, 0), sticky="e")
tk.Button(btns, text="Cancel", command=self.destroy).pack(side="right", padx=(6, 0))
tk.Button(btns, text="Save", command=self._save).pack(side="right")

def _rotate_device_key(self) -> None:
    if not messagebox.askyesno(
        "Rotate device key",
        "This re-encrypts every folder's stored face template under a new local key.\n"
        "It does not change any enrolled face or require re-enrollment.\nContinue?",
        parent=self,
    ):
        return
    try:
        count = face_auth.rotate_device_key_all_profiles()
        messagebox.showinfo(
            "Done",
            f"Device key rotated. {count} folder face template(s) re-encrypted.",
            parent=self,
        )
    except Exception as exc:
        messagebox.showerror("Error", _safe_error_text(exc), parent=self)

def _save(self) -> None:
    config.save_settings({
        "face_match_threshold": float(self.threshold_var.get()),
        "face_verify_timeout_seconds": int(self.timeout_var.get()),
        "liveness_check_enabled": bool(self.liveness_var.get()),
        "restrict_folder_access": bool(self.restrict_var.get()),
        "argon2_time_cost": int(self.time_cost_var.get()),
        "argon2_memory_cost_kib": int(self.mem_cost_var.get()),
        "argon2_parallelism": int(self.parallelism_var.get()),
    })
    messagebox.showinfo("Saved", "Settings saved.", parent=self)
    self.destroy()

# -----
# New-password dialog (used by Change Password)
# -----

class NewPasswordDialog(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Set New Password")
        self.resizable(False, False)
        self.transient(master)
        self.grab_set()
        self.new_password: Optional[str] = None

    frm = tk.Frame(self, padx=20, pady=20)
    frm.pack()

```



```

tk.Label(frm, text="New password:").pack(anchor="w")
pw_var = tk.StringVar()
tk.Entry(frm, textvariable=pw_var, show="*", width=34).pack()

tk.Label(frm, text="Confirm new password:").pack(anchor="w", pady=(8, 0))
pw2_var = tk.StringVar()
tk.Entry(frm, textvariable=pw2_var, show="*", width=34).pack()

checklist = PasswordChecklist(frm)
checklist.pack(fill="x", pady=(10, 0))
match_label = tk.Label(frm, text="", fg="#b00020")
match_label.pack(anchor="w")

ok_btn = tk.Button(frm, text="Change Password", state="disabled")
ok_btn.pack(anchor="e", pady=(12, 0))

def on_change(* a):
    valid = checklist.update_from(pw_var.get())
    match = pw_var.get() == pw2_var.get() and pw_var.get() != ""
    match_label.config(text="" if match else "Passwords do not match.")
    ok_btn.config(state="normal" if (valid and match) else "disabled")

pw_var.trace_add("write", on_change)
pw2_var.trace_add("write", on_change)

def confirm():
    self.new_password = pw_var.get()
    pw_var.set("")
    pw2_var.set("")
    self.destroy()

ok_btn.config(command=confirm)
self.protocol("WM_DELETE_WINDOW", self.destroy)

```

```

# -----
# Folder pickers
# -----

```

```

class FolderPicker(tk.Toplevel):
    """Chooses among folders this app knows about.

```

A picker rather than a browse dialog because a locked folder is access-restricted: Explorer cannot open it, so there is nothing to confirm by browsing into it. `selected` is the chosen Path, or None.

```

def __init__(self, master, entries, title="Select Folder",
              action_label="Select", allow_browse=False, empty_message=None):
    super().__init__(master)
    self.title(title)
    self.resizable(False, False)
    self.transient(master)
    self.grab_set()
    self.selected = None
    self._entries = list(entries)

    frm = tk.Frame(self, padx=16, pady=16)
    frm.pack(fill="both", expand=True)

    tk.Label(frm, text=title + ":", font=("Segoe UI", 11, "bold")).pack(anchor="w")

    list_frame = tk.Frame(frm)
    list_frame.pack(fill="both", expand=True, pady=(6, 0))
    scrollbar = tk.Scrollbar(list_frame, orient="vertical")
    self.listbox = tk.Listbox(
        list_frame, width=68, height=8, yscrollcommand=scrollbar.set, font=("Consolas", 9)
    )
    scrollbar.config(command=self.listbox.yview)
    scrollbar.pack(side="right", fill="y")
    self.listbox.pack(side="left", fill="both", expand=True)
    self.listbox.bind("<Double-Button-1>", lambda _e: self._choose())

    if self._entries:
        for entry in self._entries:
            bits = []
            if entry.get("locked"):
                bits.append("LOCKED")
            if not entry.get("exists", True):
                bits.append("MISSING")
            suffix = ("[" + ", ".join(bits) + "]" if bits else "")
            self.listbox.insert("end", entry["path"] + suffix)
        self.listbox.selection_set(0)
    else:

```

```

        self.listbox.insert("end", empty_message or "(nothing to show)")
        self.listbox.config(state="disabled")

    tk.Label(
        frm,
        text="Each folder has its own password and its own enrolled face.",
        fg="#555555", justify="left",
    ).pack(anchor="w", pady=(8, 0))

    btns = tk.Frame(frm)
    btns.pack(fill="x", pady=(12, 0))
    tk.Button(btns, text="Cancel", command=self._cancel).pack(side="right")
    tk.Button(btns, text=action_label, command=self._choose).pack(side="right", padx=(0, 6))
    if allow_browse:
        tk.Button(btns, text="Browse...", command=self._browse).pack(side="left")
    if any(not e.get("exists", True) for e in self._entries):
        tk.Button(btns, text="Forget Missing", command=self._forget_missing).pack(side="left", padx=(6, 0))

    self.protocol("WM_DELETE_WINDOW", self._cancel)

def _choose(self):
    if not self._entries:
        return
    sel = self.listbox.curselection()
    if not sel:
        messagebox.showinfo("Select a folder", "Please select a folder from the list.", parent=self)
        return
    entry = self._entries[sel[0]]
    if not entry.get("exists", True):
        messagebox.showerror(
            "Folder missing",
            "This folder no longer exists at that location. It may have been "
            "moved or deleted outside this app." + NL + NL + "Use 'Forget Missing' to remove it.",
            parent=self,
        )
    return
    self.selected = Path(entry["path"])
    self.destroy()

def _browse(self):
    chosen = filedialog.askdirectory(title="Select folder", parent=self)
    if chosen:
        self.selected = Path(chosen)
        self.destroy()

def _forget_missing(self):
    missing = [e for e in self._entries if not e.get("exists", True)]
    if not missing:
        messagebox.showinfo("Nothing to forget", "All listed folders still exist.", parent=self)
        return
    if not messagebox.askyesno(
        "Forget missing folders",
        "Remove " + str(len(missing)) + " missing folder(s) from the list?" + NL + NL
        + "This only updates this app's records. No files are deleted, but the "
        "stored password and face for those folders are discarded.",
        parent=self,
    ):
        return
    for entry in missing:
        folder_manager.forget_folder(Path(entry["path"]), delete_profile=True)
    messagebox.showinfo("Updated", "Missing folders removed from the list.", parent=self)
    self.destroy()

def _cancel(self):
    self.selected = None
    self.destroy()

class LockedFolderPicker(FolderPicker):
    """Convenience wrapper listing only the folders currently locked."""

    def __init__(self, master):
        super().__init__(
            master,
            folder_manager.list_locked_folders(),
            title="Locked folders",
            action_label="Unlock Selected",
            allow_browse=True,
            empty_message="(no folders are currently locked)",
        )

# -----
# Main application window
# -----

```

```

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(config.APP_NAME)
        self.resizable(False, False)
        config.ensure_dirs()

        self.status_var = tk.StringVar(value="Ready.")

        header = tk.Label(self, text="Folder Lock", font=("Segoe UI", 18, "bold"), pady=16)
        header.pack()

        btn_frame = tk.Frame(self, padx=24, pady=8)
        btn_frame.pack()

        buttons = [
            ("Lock a Folder", self._on_lock_folder),
            ("Unlock a Folder", self._on_unlock_folder),
            ("Change a Folder's Password", self._on_change_password),
            ("Re-enroll a Folder's Face", self._on_enroll_face),
            ("Security Settings", self._on_security_settings),
            ("Exit", self.destroy),
        ]
        for text, cmd in buttons:
            tk.Button(btn_frame, text=text, width=30, command=cmd).pack(pady=4)

        tk.Label(
            self,
            text="Every folder has its own password and its own enrolled face.",
            fg="#555555", pady=(6),
        ).pack()

        status_bar = tk.Label(self, textvariable=self.status_var, bd=1, relief="sunken", anchor="w", padx=8)
        status_bar.pack(fill="x", side="bottom")

# -- state / status helpers -----
def _set_status(self, text: str) -> None:
    self.status_var.set(text)

def _pick_known_folder(self, title: str, action_label: str) -> Optional[Path]:
    """Choose among folders that already have their own credentials."""
    entries = [e for e in folder_manager.list_known_folders() if e.get("profile_id")]
    if not entries:
        messagebox.showinfo(
            "No protected folders",
            "No folder has been set up yet. Use 'Lock a Folder' to protect one.",
            parent=self,
        )
        return None
    picker = FolderPicker(self, entries, title=title, action_label=action_label)
    self.wait_window(picker)
    return picker.selected

def _authenticate_for(self, folder_path: Path, title: str) -> Optional[bytearray]:
    """Runs the mandatory face-then-password check against THIS folder's
    own credentials. Returns that folder's master key, or None if denied."""
    profile_id = folder_manager.profile_id_for(folder_path)
    if not profile_id or not folder_manager.has_credentials(folder_path):
        messagebox.showerror(
            "No credentials",
            "This app has no stored credentials for that folder, so it "
            "cannot be unlocked here.",
            parent=self,
        )
        return None
    dialog = AuthDialog(self, profile_id, mode="full", title=title)
    self.wait_window(dialog)
    if not dialog.result:
        self._set_status(Status.ACCESS_DENIED)
        return None
    return dialog.result

# -- menu actions -----
def _on_lock_folder(self) -> None:
    folder = filedialog.askdirectory(title="Select a folder to lock")
    if not folder:
        return
    folder_path = Path(folder)
    if folder_manager.is_locked(folder_path):
        messagebox.showerror("Cannot lock", "This folder is already locked.", parent=self)
        return

    if folder_manager.has_credentials(folder_path):
        # This folder has been protected before: prove ownership with its
        # own face and password rather than silently re-protecting it.

```

```

        master_key = self._authenticate_for(
            folder_path, f"Authenticate to Lock '{folder_path.name}'"
        )
        if not master_key:
            return
        profile_id = folder_manager.profile_id_for(folder_path)
    else:
        profile_id = folder_manager.new_profile_id()
        wizard = FolderCredentialsWizard(self, profile_id, folder_path)
        self.wait_window(wizard)
        if not wizard.success or wizard.master_key is None:
            self._set_status("Setup cancelled - folder was not locked.")
            return
        master_key = wizard.master_key
        folder_manager.assign_profile(folder_path, profile_id)

def verify_restriction() -> None:
    """Confirm the folder really is closed off, rather than assuming
    it. If encryption succeeded but access restriction silently did
    not apply, the user must be told plainly - otherwise they would
    believe the folder is inaccessible when it is merely encrypted."""
    settings = config.get_settings()
    if not settings.get("restrict_folder_access", True):
        return
    if not access_control.is_available():
        messagebox.showwarning(
            "Encrypted, but access not restricted",
            "The folder's contents are encrypted, but blocking the folder "
            "from opening is only supported on Windows (NTFS).",
            parent=self,
        )
        return
    if not access_control.is_access_denied(folder_path):
        messagebox.showwarning(
            "Encrypted, but access not restricted",
            "The folder's contents are encrypted and unreadable, but Windows "
            "did not apply the access restriction - so the folder can still "
            "be opened.\n\nThis usually means the drive is not NTFS-formatted "
            "(for example a FAT32/exFAT USB drive).",
            parent=self,
        )

self._run_folder_task(
    folder_manager.lock_folder,
    folder_path,
    master_key,
    profile_id,
    success_status=Status.ENCRYPTION_COMPLETED,
    failure_status=Status.ENCRYPTION_FAILED,
    sensitive_key=master_key,
    on_success=verify_restriction,
)

def _on_unlock_folder(self) -> None:
    picker = LockedFolderPicker(self)
    self.wait_window(picker)
    folder_path = picker.selected
    if not folder_path:
        return
    if not folder_manager.is_locked(folder_path):
        messagebox.showerror("Not a locked folder", "This folder is not locked.", parent=self)
        return

    master_key = self._authenticate_for(
        folder_path, f"Authenticate to Unlock '{folder_path.name}'"
    )
    if not master_key:
        return

    self._run_folder_task(
        folder_manager.unlock_folder,
        folder_path,
        master_key,
        success_status=Status.FOLDER_UNLOCKED,
        failure_status=Status.DECRYPTION_FAILED,
        sensitive_key=master_key,
    )

def _on_enroll_face(self) -> None:
    folder_path = self._pick_known_folder(
        "Re-enroll Face for a Folder", "Re-enroll Face"
    )
    if not folder_path:
        return
    if not messagebox.askyesno(
        "Re-enroll face",
        f"Changing the face for '{folder_path.name}' requires verifying its "
        "CURRENT face and password first.\n\nOther folders are unaffected. Continue?",
        parent=self,
    ):

```

```

        return
    master_key = self._authenticate_for(
        folder_path, f"Authenticate to Re-enroll Face for '{folder_path.name}'"
    )
    if not master_key:
        return
    security.best_effort_zero(master_key)

    profile_id = folder_manager.profile_id_for(folder_path)
    dlg = EnrollmentDialog(self, profile_id, title=f"Re-enroll Face - {folder_path.name}")
    self.wait_window(dlg)
    self._set_status(
        f"Face re-enrolled for '{folder_path.name}'." if dlg.success else "Re-enrollment cancelled."
    )

def _on_change_password(self) -> None:
    folder_path = self._pick_known_folder(
        "Change Password for a Folder", "Change Password"
    )
    if not folder_path:
        return
    master_key = self._authenticate_for(
        folder_path, f"Authenticate to Change Password for '{folder_path.name}'"
    )
    if not master_key:
        return

    try:
        dlg = NewPasswordDialog(self)
        self.wait_window(dlg)
        if not dlg.new_password:
            self._set_status("Password change cancelled.")
            return
        profile_id = folder_manager.profile_id_for(folder_path)
        password_auth.create_password_vault(profile_id, dlg.new_password, master_key)
        messagebox.showinfo(
            "Success",
            f"Password changed for '{folder_path.name}'.\n\nOther folders keep their own passwords.",
            parent=self,
        )
        self._set_status(f"Password changed for '{folder_path.name}'")
    except Exception as exc:
        messagebox.showerror("Error", _safe_error_text(exc), parent=self)
    finally:
        security.best_effort_zero(master_key)

def _on_security_settings(self) -> None:
    SecuritySettingsDialog(self)

# -- generic background folder task runner -----

def _run_folder_task(
    self, fn, *args, success_status: str, failure_status: str,
    sensitive_key: Optional[bytearray] = None, on_success=None,
) -> None:
    progress_win = tk.Toplevel(self)
    progress_win.title("Please wait")
    progress_win.resizable(False, False)
    progress_win.transient(self)
    progress_win.grab_set()
    # Encryption/decryption is running on disk in the background; closing
    # this window would only hide progress, not stop the operation, and
    # would leave the user with no way to see the final result. Disable
    # the close button for the duration instead of allowing a confusing
    # half-cancelled state.
    progress_win.protocol("WM_DELETE_WINDOW", lambda: None)

    label = tk.Label(progress_win, text="Starting...", width=60, anchor="w", padx=12, pady=12)
    label.pack()
    pbar = ttk.Progressbar(progress_win, mode="indeterminate", length=360)
    pbar.pack(padx=12, pady=(0, 12))
    pbar.start(15)

    q: "queue.Queue" = queue.Queue()

    def worker() -> None:
        try:
            fn(*args, progress_cb=lambda m: q.put(("status", m)))
            q.put(("done", None))
        except Exception as exc: # surfaced below, not swallowed
            q.put(("error", exc))
        finally:
            # Only safe to wipe the key here: this runs after fn() has
            # fully returned (success or failure), so nothing is still
            # using it. Zeroing it any earlier – e.g. right after
            # starting this thread – would race with the encryption/
            # decryption still in progress.
            if sensitive_key is not None:
                security.best_effort_zero(sensitive_key)

```

```

threading.Thread(target=worker, daemon=True).start()

def poll() -> None:
    if not progress_win.wininfo_exists():
        return
    try:
        while True:
            kind, payload = q.get_nowait()
            if kind == "status":
                label.config(text=payload)
            elif kind == "done":
                pbar.stop()
                progress_win.destroy()
                self._set_status(success_status)
                messagebox.showinfo("Success", success_status, parent=self)
                if on_success is not None:
                    on_success()
                return
            elif kind == "error":
                pbar.stop()
                progress_win.destroy()
                self._set_status(failure_status)
                messagebox.showerror(failure_status, _safe_error_text(payload), parent=self)
                return
    except queue.Empty:
        pass
    progress_win.after(60, poll)

progress_win.after(60, poll)

```

## 6.2 Build and Dependencies

### requirements.txt

*Runtime dependencies.*

16 lines

```

# Cryptography: Argon2id KDF + AES-256-GCM AEAD (via OpenSSL bindings)
argon2-cffi>=23.1.0
cryptography>=42.0.0

# Face detection (Haar cascade) + face recognition (LBPH), both built into
# OpenCV's contrib module. NOTE: this must be opencv-contrib-python, not
# plain opencv-python - the base package does not include cv2.face. Do not
# install both packages in the same environment (they conflict); if you
# have opencv-python installed, remove it first:
# pip uninstall opencv-python -y
opencv-contrib-python>=4.9.0

# Camera capture + image handling
numpy>=1.24.0
Pillow>=10.0.0

```

### build\_exe.py

*PyInstaller build script.*

106 lines

```

"""
build_exe.py
-----
Builds a standalone Windows .exe for Folder Lock using PyInstaller.

This is a BUILD-TIME script, not something end users run. It must be run
in an environment that already has every runtime dependency installed
(see requirements.txt) - PyInstaller can only bundle what is actually
importable in the environment it runs in. `bootstrap.py`'s first-run
auto-installer is for end users running the app from source
(`python main.py`); once frozen into an .exe by this script, bootstrap.py
detects it is running frozen and does nothing (see bootstrap._is_frozen()).

Usage:
    pip install -r requirements.txt
    pip install pyinstaller
    python build_exe.py

```

```

Output:
dist/FolderLock/FolderLock.exe (a --onedir build – more reliable
than --onefile for OpenCV's native binaries, and starts faster).
"""

from __future__ import annotations

import shutil
import subprocess
import sys
from pathlib import Path

APP_NAME = "FolderLock"
ENTRY_POINT = "main.py"

def main() -> None:
    try:
        import PyInstaller # noqa: F401
    except ImportError:
        print("PyInstaller is not installed. Run: pip install pyinstaller", file=sys.stderr)
        sys.exit(1)

    missing_runtime = []
    for module_name in ("cryptography", "argon2", "numpy", "PIL", "cv2"):
        try:
            __import__(module_name)
        except ImportError:
            missing_runtime.append(module_name)
    if missing_runtime:
        print(
            "Cannot build: the following runtime packages are not installed "
            f"in THIS environment: {' '.join(missing_runtime)}\n"
            "Install them first (pip install -r requirements.txt), then re-run this script.",
            file=sys.stderr,
        )
        sys.exit(1)

    import cv2
    if not hasattr(cv2, "face") or not hasattr(cv2.face, "LBPHFaceRecognizer_create"):
        print(
            "Cannot build: the installed OpenCV is missing the 'face' (contrib) module.\n"
            "Install the contrib build: pip uninstall opencv-python -y && "
            "pip install opencv-contrib-python",
            file=sys.stderr,
        )
        sys.exit(1)

    project_dir = Path(__file__).resolve().parent
    resources_dir = project_dir / "resources"
    if not (resources_dir / "haarcascade_frontalface_default.xml").exists():
        print(f"Cannot build: missing {resources_dir}/haarcascade_frontalface_default.xml", file=sys.stderr)
        sys.exit(1)

    dist_dir = project_dir / "dist"
    build_dir = project_dir / "build"
    spec_file = project_dir / f"{APP_NAME}.spec"
    for stale in (dist_dir, build_dir):
        if stale.exists():
            shutil.rmtree(stale)
    if spec_file.exists():
        spec_file.unlink()

    cmd = [
        sys.executable, "-m", "PyInstaller",
        "--name", APP_NAME,
        "--windowed", # GUI app: no console window
        "--onedir", # reliable for OpenCV's native libs
        "--noconfirm",
        "--collect-all", "cv2",
        "--hidden-import", "PIL._tkinter_finder",
        "--add-data", f"{resources_dir}{';' if sys.platform == 'win32' else ':'}resources",
        str(project_dir / ENTRY_POINT),
    ]
    print("Running:", " ".join(cmd))
    result = subprocess.run(cmd, cwd=str(project_dir))
    if result.returncode != 0:
        print("PyInstaller build failed.", file=sys.stderr)
        sys.exit(result.returncode)

    exe_path = dist_dir / APP_NAME / f"{APP_NAME}.exe"
    print(f"\nBuild complete: {exe_path}")
    print("Copy the whole 'dist/FolderLock' folder when distributing – the .exe depends on the files next to it.")

if __name__ == "__main__":

```

```
main()
```

## 6.3 Test Suites

### \_selftest.py

*Cryptography, password policy and folder lock/unlock.*

178 lines

```
"""
Ad-hoc self-test (not part of the shipped app) exercising the non-camera
crypto/file layers: password validation, master-key wrapping, streaming
file encryption/decryption round-trip, tamper detection, and a full
folder lock/unlock cycle. Run manually: python _selftest.py
"""
import shutil
import tempfile
from pathlib import Path

import atexit

import access_control
import config
import encryption
import folder_manager
import password_auth
import security

tmp_root = Path(tempfile.mkdtemp(prefix="flock_selftest_"))

@atexit.register
def _guaranteed_cleanup() -> None:
    """A test that dies part-way through a lock would otherwise leave an
    access-restricted temp folder behind that rmtree cannot even enumerate.
    Always lift any restriction we applied before deleting."""
    try:
        for child in tmp_root.rglob("*"):
            if child.is_dir():
                try:
                    access_control.restore_access(child)
                except Exception:
                    pass
            access_control.restore_access(tmp_root)
    except Exception:
        pass
    shutil.rmtree(tmp_root, ignore_errors=True)
config.APP_DATA_DIR = tmp_root / "appdata"
config.VAULT_DIR = config.APP_DATA_DIR / "vault"
config.LOG_DIR = config.APP_DATA_DIR / "logs"
config.DEVICE_KEY_FILE = config.VAULT_DIR / "device.key"
config.FACE_TEMPLATE_FILE = config.VAULT_DIR / "face_template.enc"
config.PASSWORD_VAULT_FILE = config.VAULT_DIR / "password_vault.json"
config.SETTINGS_FILE = config.VAULT_DIR / "settings.json"
config.ensure_dirs()

print("== password validation ==")
for pw, expect_ok in [
    ("123456", False),
    ("abcdef", False),
    ("abc123", False),
    ("abcdef!", False),
    ("Abc1!x", True),
    ("Secure9@", True),
    ("MyLock#7", True),
]:
    ok, problems = password_auth.validate_password(pw)
    status = "OK" if ok == expect_ok else "FAIL"
    print(f" {status}: {pw!r} -> valid={ok} problems={problems}")
    assert ok == expect_ok, f"password validation mismatch for {pw!r}"

print("== master key wrap / unwrap via password vault ==")
master_key = security.generate_master_key()
PROFILE = "test-profile"
password_auth.create_password_vault(PROFILE, "Correct#123", master_key)
recovered = password_auth.verify_password_and_get_master_key(PROFILE, "Correct#123")
assert recovered == master_key, "recovered master key mismatch"
print(" OK: correct password recovers exact master key")

try:
    password_auth.verify_password_and_get_master_key(PROFILE, "Wrong#12345")
```



```

    raise SystemExit("FAIL: wrong password did not raise")
except security.AuthenticationError:
    print(" OK: wrong password rejected (AuthenticationError)")

print("== best_effort_zero actually wipes the real key (not a throwaway copy) ==")
key_to_wipe = password_auth.verify_password_and_get_master_key(PROFILE, "Correct#123")
assert isinstance(key_to_wipe, bytearray), "keys must be mutable bytearray so they can genuinely be zeroed"
assert any(b != 0 for b in key_to_wipe), "sanity check: key should not already be all zero"
security.best_effort_zero(key_to_wipe)
assert all(b == 0 for b in key_to_wipe), "best_effort_zero did not actually wipe the real key buffer"
print(" OK: the actual key buffer is zeroed in place, not a disposable bytearray() copy")

print("== change password re-wraps same master key ==")
password_auth.change_password(PROFILE, "Correct#123", "NewPass#456")
recovered2 = password_auth.verify_password_and_get_master_key(PROFILE, "NewPass#456")
assert recovered2 == master_key
print(" OK: master key unchanged after password change")

print("== streaming file encryption round-trip (multi-chunk) ==")
config.save_settings({"chunk_size_bytes": 1024}) # force multiple small chunks
work = tmp_root / "work"
work.mkdir()
plain = work / "plain.bin"
plain.write_bytes(bytes((i % 256) for i in range(5000)))
enc = work / "plain.bin.enc"
dec = work / "plain.bin.dec"
encryption.encrypt_file(plain, enc, master_key, b"selftest/plain.bin")
encryption.decrypt_file(enc, dec, master_key, b"selftest/plain.bin")
assert dec.read_bytes() == plain.read_bytes()
print(f" OK: {plain.stat().st_size} bytes round-tripped across multiple chunks")

print("== zero-byte file round-trip ==")
empty = work / "empty.bin"
empty.write_bytes(b"")
enc0 = work / "empty.bin.enc"
dec0 = work / "empty.bin.dec"
encryption.encrypt_file(empty, enc0, master_key, b"selftest/empty.bin")
encryption.decrypt_file(enc0, dec0, master_key, b"selftest/empty.bin")
assert dec0.read_bytes() == b""
print(" OK: zero-byte file round-tripped")

print("== tamper detection ==")
tampered = bytearray(enc.read_bytes())
tampered[-1] ^= 0xFF # flip a bit in the last chunk's ciphertext/tag
enc_tampered = work / "plain.bin.tampered.enc"
enc_tampered.write_bytes(bytes(tampered))
try:
    encryption.decrypt_file(enc_tampered, work / "should_not_exist.dec", master_key, b"selftest/plain.bin")
    raise SystemExit("FAIL: tampered ciphertext decrypted without error")
except security.TamperDetectedError:
    print(" OK: bit-flip in ciphertext detected (TamperDetectedError)")
assert not (work / "should_not_exist.dec").exists(), "FAIL: partial plaintext leaked to disk after tamper"
print(" OK: no partial plaintext file was left on disk")

print("== wrong AAD (simulated path-rename attack) detected ==")
try:
    encryption.decrypt_file(enc, work / "should_not_exist2.dec", master_key, b"selftest/DIFFERENT_PATH.bin")
    raise SystemExit("FAIL: wrong AAD decrypted without error")
except security.TamperDetectedError:
    print(" OK: mismatched associated data (moved/renamed file) detected")

print("== full folder lock/unlock cycle (folder name/path must never change) ==")
protected = tmp_root / "MyDocs"
(protected / "sub").mkdir(parents=True)
(protected / "a.txt").write_text("hello world")
(protected / "sub" / "b.bin").write_bytes(bytes(range(256)) * 20)
(protected / "sub" / "empty_dir").mkdir()

assert not folder_manager.is_locked(protected)

events = []
result_path = folder_manager.lock_folder(protected, master_key, PROFILE, progress_cb=events.append)
assert result_path == protected, "lock_folder must return the SAME path, not a renamed one"
assert protected.exists() and protected.is_dir(), "folder must still exist at its original name/path after locking"
assert folder_manager.is_locked(protected), "folder should now report as locked"
assert (protected / "a.txt.enc").exists(), "plaintext file should be replaced by its encrypted form"
assert not (protected / "a.txt").exists(), "plaintext file should no longer exist in cleartext"
print(f" OK: locked in place - still named '{protected.name}' at the same path ({len(events)} progress events)")

restored_path = folder_manager.unlock_folder(protected, master_key, progress_cb=events.append)
assert restored_path == protected, "unlock_folder must return the SAME path"
assert not folder_manager.is_locked(protected), "folder should no longer report as locked"
assert (protected / "a.txt").read_text() == "hello world"
assert (protected / "sub" / "b.bin").read_bytes() == bytes(range(256)) * 20
assert (protected / "sub" / "empty_dir").is_dir()
print(" OK: unlocked in place, file contents + empty directory structure preserved, name unchanged throughout")

print("== lock refuses to double-lock an already-locked folder ==")

```

```

folder_manager.lock_folder(protected, master_key, PROFILE, progress_cb=lambda m: None)
try:
    folder_manager.lock_folder(protected, master_key, PROFILE, progress_cb=lambda m: None)
    raise SystemExit("FAIL: locking an already-locked folder did not raise")
except folder_manager.FolderManagerError:
    print(" OK: refused to lock an already-locked folder")

print("== unlock refuses to unlock a folder that isn't locked ==")
folder_manager.unlock_folder(protected, master_key, progress_cb=lambda m: None) # back to plaintext
try:
    folder_manager.unlock_folder(protected, master_key, progress_cb=lambda m: None)
    raise SystemExit("FAIL: unlocking a non-locked folder did not raise")
except folder_manager.FolderManagerError:
    print(" OK: refused to unlock a folder that isn't locked")

shutil.rmtree(tmp_root, ignore_errors=True)
print("\nALL SELF-TESTS PASSED")

```

## \_selftest\_multi.py

*Per-folder credential independence.*

156 lines

```

"""
Ad-hoc self-test for PER-FOLDER credentials: proves that each protected
folder has genuinely independent credentials, so that one folder's password
or face grants no access whatsoever to another folder.

```

This is the security property that matters most about the per-folder design, so it is asserted directly rather than assumed from the fact that separate files exist on disk.

Run: python \_selftest\_multi.py

```

"""
import atexit
import shutil
import sys
import tempfile
from pathlib import Path

import access_control
import config
import security

tmp_root = Path(tempfile.mkdtemp(prefix="flock_multi_selftest_"))

```

```

@atexit.register
def _cleanup() -> None:
    try:
        for child in list(tmp_root.rglob("*")) + [tmp_root]:
            if child.is_dir():
                try:
                    access_control.restore_access(child)
                except Exception:
                    pass
    except Exception:
        pass
    shutil.rmtree(tmp_root, ignore_errors=True)

```

```

config.APP_DATA_DIR = tmp_root / "appdata"
config.VAULT_DIR = config.APP_DATA_DIR / "vault"
config.LOG_DIR = config.APP_DATA_DIR / "logs"
config.DEVICE_KEY_FILE = config.VAULT_DIR / "device.key"
config.SETTINGS_FILE = config.VAULT_DIR / "settings.json"
config.ensure_dirs()

```

```

import folder_manager # noqa: E402
import password_auth # noqa: E402

```

```

# Two folders, each with its own password and its own encryption key.
folder_a = tmp_root / "ClientA"
folder_b = tmp_root / "ClientB"
folder_a.mkdir()
folder_b.mkdir()
(folder_a / "a-secret.txt").write_text("client A confidential")
(folder_b / "b-secret.txt").write_text("client B confidential")

```

```

PW_A = "AlphaPass1!"
PW_B = "BravoPass2@"

```

```

profile_a = folder_manager.new_profile_id()
profile_b = folder_manager.new_profile_id()

print("== distinct profile ids ==")
assert profile_a != profile_b
print(f" OK: {profile_a[:8]}... != {profile_b[:8]}...")

print("== each folder gets its own independent master key ==")
key_a = security.generate_master_key()
key_b = security.generate_master_key()
assert key_a != key_b, "each folder must get its own randomly generated key"
password_auth.create_password_vault(profile_a, PW_A, key_a)
password_auth.create_password_vault(profile_b, PW_B, key_b)
print(" OK: two independent keys, two independent password vaults")

print("== a folder's own password recovers only its own key ==")
got_a = password_auth.verify_password_and_get_master_key(profile_a, PW_A)
got_b = password_auth.verify_password_and_get_master_key(profile_b, PW_B)
assert got_a == key_a and got_b == key_b
assert got_a != got_b
print(" OK: each password unlocks exactly one folder's key")

print("== THE KEY PROPERTY: folder A's password must NOT open folder B ==")
try:
    password_auth.verify_password_and_get_master_key(profile_b, PW_A)
    raise SystemExit("FAIL: folder A's password was accepted for folder B")
except security.AuthenticationError:
    print(" OK: A's password rejected by B")
try:
    password_auth.verify_password_and_get_master_key(profile_a, PW_B)
    raise SystemExit("FAIL: folder B's password was accepted for folder A")
except security.AuthenticationError:
    print(" OK: B's password rejected by A")

print("== lock both folders independently ==")
folder_manager.assign_profile(folder_a, profile_a)
folder_manager.assign_profile(folder_b, profile_b)
folder_manager.lock_folder(folder_a, key_a, profile_a, progress_cb=lambda m: None)
folder_manager.lock_folder(folder_b, key_b, profile_b, progress_cb=lambda m: None)
assert folder_manager.is_locked(folder_a) and folder_manager.is_locked(folder_b)
assert folder_manager.profile_id_for(folder_a) == profile_a
assert folder_manager.profile_id_for(folder_b) == profile_b
print(" OK: both locked, each bound to its own profile")

print("== folder B's key must NOT decrypt folder A ==")
try:
    folder_manager.unlock_folder(folder_a, key_b, progress_cb=lambda m: None)
    raise SystemExit("FAIL: folder B's key decrypted folder A")
except SystemExit:
    raise
except Exception:
    print(" OK: cross-folder decryption rejected")
assert folder_manager.is_locked(folder_a), "A must remain locked after the failed attempt"

print("== unlocking A leaves B untouched ==")
folder_manager.unlock_folder(folder_a, key_a, progress_cb=lambda m: None)
assert not folder_manager.is_locked(folder_a)
assert folder_manager.is_locked(folder_b), "unlocking one folder must not unlock another"
assert (folder_a / "a-secret.txt").read_text() == "client A confidential"
print(" OK: A is open, B is still locked")

print("== changing A's password does not affect B ==")
password_auth.change_password(profile_a, PW_A, "NewAlpha9#")
assert password_auth.verify_password_and_get_master_key(profile_a, "NewAlpha9#") == key_a
assert password_auth.verify_password_and_get_master_key(profile_b, PW_B) == key_b
try:
    password_auth.verify_password_and_get_master_key(profile_b, "NewAlpha9#")
    raise SystemExit("FAIL: A's new password works on B")
except security.AuthenticationError:
    pass
print(" OK: B's password is unchanged and still exclusively B's")

print("== unlock B with its own credentials ==")
folder_manager.unlock_folder(folder_b, key_b, progress_cb=lambda m: None)
assert (folder_b / "b-secret.txt").read_text() == "client B confidential"
print(" OK: B recovered with its own key")

print("== registry tracks both folders separately ==")
known = {e["path"]: e for e in folder_manager.list_known_folders()}
assert str(folder_a) in known and str(folder_b) in known
assert known[str(folder_a)]["profile_id"] != known[str(folder_b)]["profile_id"]
print(" OK: two folders, two profiles, tracked independently")

print("== deleting a profile is refused while it protects a locked folder ==")
folder_manager.lock_folder(folder_b, key_b, profile_b, progress_cb=lambda m: None)
try:

```

```

        folder_manager.delete_profile(profile_b)
        raise SystemExit("FAIL: deleted credentials still protecting a locked folder")
    except SystemExit:
        raise
    except folder_manager.FolderManagerError:
        print(" OK: refused to delete in-use credentials")
    folder_manager.unlock_folder(folder_b, key_b, progress_cb=lambda m: None)

print("\nALL PER-FOLDER CREDENTIAL SELF-TESTS PASSED")
sys.exit(0)

```

## \_selftest\_face.py

*Face recognition mechanics.*

102 lines

```

"""
Ad-hoc self-test for face_auth.py's non-camera mechanics: Haar cascade
loading, and the LBPH train -> serialize -> encrypt -> decrypt -> restore
-> predict round-trip through the same encrypted-template storage path the
real app uses. Uses synthetic grayscale images (not real faces) since this
only needs to validate the mechanical pipeline, not detection accuracy -
detection/recognition accuracy requires a live camera and is covered by
manual testing.

Run inside an environment that actually has opencv-contrib-python (cv2.face)
available, e.g. the conda env created for this build:
conda run -n folderlock_build python _selftest_face.py
"""
import shutil
import tempfile
from pathlib import Path

import numpy as np

import config

tmp_root = Path(tempfile.mkdtemp(prefix="flock_face_selftest_"))
config.APP_DATA_DIR = tmp_root / "appdata"
config.VAULT_DIR = config.APP_DATA_DIR / "vault"
config.LOG_DIR = config.APP_DATA_DIR / "logs"
config.DEVICE_KEY_FILE = config.VAULT_DIR / "device.key"
config.PASSWORD_VAULT_FILE = config.VAULT_DIR / "password_vault.json"
config.SETTINGS_FILE = config.VAULT_DIR / "settings.json"
config.ensure_dirs()

import face_auth # noqa: E402 (import after config override, matches _selftest.py pattern)

print("== dependency check ==")
face_auth.require_dependencies()
print(" OK: cv2 + cv2.face are importable and usable")

print("== Haar cascade loading ==")
detector = face_auth.Detector()
assert not detector.face_cascade.empty()
assert not detector.eye_cascade.empty()
print(" OK: bundled face + eye cascades load from resources/")

print("== is_enrolled() before enrollment ==")
PROFILE = "test-face-profile"
fa = face_auth.FaceAuthenticator(PROFILE)
assert fa.is_enrolled() is False
print(" OK: not enrolled in a fresh app-data dir")

print("== LBPH train / encrypted-save / encrypted-load / predict round-trip ==")

def _synthetic_face(seed: int) -> np.ndarray:
    """A deterministic, seed-dependent 200x200 grayscale pattern. Not a real
    face - only used to exercise LBPH's train/predict mechanics and the
    encrypted-template storage path, not real recognition accuracy."""
    rng = np.random.default_rng(seed)
    base = rng.integers(0, 256, size=face_auth.FACE_SIZE, dtype=np.uint8)
    return base

import cv2 # noqa: E402

# "Enrollment": train directly (bypassing the camera loop) on 5 samples of
# "person A" and save through the real encrypted-storage path.

```

```

person_a_samples = [_synthetic_face(seed=100 + i) for i in range(5)]
recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.train(person_a_samples, np.array([0] * len(person_a_samples)))
fa._save_recognizer(recognizer)

assert fa.template_path.exists()
assert fa.is_enrolled() is True
raw = fa.template_path.read_bytes()
assert raw[:20] != b"<?xml"[:20], "template must not be stored as plaintext XML"
print(f" OK: encrypted template written ({len(raw)} bytes), not plaintext on disk")

# "Verification": load the recognizer back through the real encrypted path
# and confirm it recognizes a sample close to what it was trained on, and
# rejects something far away.
loaded = fa._load_recognizer()
label, confidence = loaded.predict(person_a_samples[0])
print(f" same-person sample -> label={label}, confidence={confidence:.2f}")
assert label == 0

different_person = _synthetic_face(seed=999999)
label2, confidence2 = loaded.predict(different_person)
print(f" different (random) sample -> label={label2}, confidence={confidence2:.2f}")
assert confidence2 > confidence, "a clearly different sample should score a higher (worse) LBPH distance"

print("== rotate_device_key() ==")
old_device_key = config.DEVICE_KEY_FILE.read_bytes()
face_auth.rotate_device_key_all_profiles()
new_device_key = config.DEVICE_KEY_FILE.read_bytes()
assert new_device_key != old_device_key
reloaded = fa._load_recognizer()
label3, _ = reloaded.predict(person_a_samples[0])
assert label3 == 0
print(" OK: device key rotated, template still decrypts and predicts correctly")

shutil.rmtree(tmp_root, ignore_errors=True)
print("\nALL FACE-AUTH MECHANICS SELF-TESTS PASSED")
print("(Real face detection/recognition accuracy still needs a live-camera manual test.)")

```

## \_selftest\_access.py

*Access restriction and recoverability.*

153 lines

```

"""
Ad-hoc self-test for the access-restriction layer: proves that a locked
folder genuinely cannot be opened (not merely that its contents are
encrypted), and – just as importantly – that access is always fully
restored afterwards so nobody can be stranded out of their own data.

Runs entirely inside a throwaway temp directory. Every path it touches is
one it created itself.

Run: python _selftest_access.py
"""
import os
import shutil
import sys
import tempfile
from pathlib import Path

import config

tmp_root = Path(tempfile.mkdtemp(prefix="flock_access_selftest_"))
config.APP_DATA_DIR = tmp_root / "appdata"
config.VAULT_DIR = config.APP_DATA_DIR / "vault"
config.LOG_DIR = config.APP_DATA_DIR / "logs"
config.DEVICE_KEY_FILE = config.VAULT_DIR / "device.key"
config.FACE_TEMPLATE_FILE = config.VAULT_DIR / "face_template.enc"
config.PASSWORD_VAULT_FILE = config.VAULT_DIR / "password_vault.json"
config.SETTINGS_FILE = config.VAULT_DIR / "settings.json"
config.ensure_dirs()

import access_control # noqa: E402
import folder_manager # noqa: E402
import security # noqa: E402

protected = tmp_root / "ClientFiles"
(protected / "sub").mkdir(parents=True)
(protected / "invoice.txt").write_text("confidential invoice")
(protected / "sub" / "notes.txt").write_text("private notes")

PROFILE = "access-test-profile"

```

```

master_key = security.generate_master_key()

def _can_open(path: Path) -> bool:
    try:
        os.listdir(path)
        return True
    except PermissionError:
        return False
    except OSError:
        return False

def _cleanup_and_exit(code: int) -> None:
    # Never leave a deny ACE behind on a test folder, even on failure.
    try:
        if protected.exists():
            access_control.restore_access(protected)
    except Exception:
        pass
    shutil.rmtree(tmp_root, ignore_errors=True)
    sys.exit(code)

try:
    print(f"== platform: {sys.platform} | access restriction available: {access_control.is_available()} ==")

    print("== before locking: folder opens normally ==")
    assert _can_open(protected), "test folder should be readable before locking"
    print("- OK: folder opens, contents visible")

    print("== locking ==")
    folder_manager.lock_folder(protected, master_key, PROFILE, progress_cb=lambda m: None)
    assert folder_manager.is_locked(protected), "folder should report as locked"
    print(" OK: locked, registry updated")

    if access_control.is_available():
        print("== THE ACTUAL REQUIREMENT: locked folder must not open ==")
        opened = _can_open(protected)
        assert not opened, "BUG: locked folder can still be opened/listed"
        print(" OK: folder REFUSES to open (PermissionError) while locked")

        print("== registry records that access was restricted ==")
        entry = folder_manager._registry_entry(protected)
        assert entry is not None and entry.get("access_restricted") is True
        print(" OK: registry entry records access_restricted=True")

        print("== lock state is still detectable even though folder is unreadable ==")
        assert folder_manager.is_locked(protected), "is_locked must not depend on reading the folder"
        print(" OK: is_locked() works via the registry, not by reading inside")

        print("== a manual recovery command is always available to the user ==")
        cmd = access_control.manual_restore_command(protected)
        assert cmd and "icaccls" in cmd and "/remove:d" in cmd
        print(f" OK: {cmd}")
    else:
        print(" SKIPPED: access restriction is Windows-only; encryption still applies")

    print("== unlocking restores access AND decrypts ==")
    folder_manager.unlock_folder(protected, master_key, progress_cb=lambda m: None)
    assert _can_open(protected), "BUG: folder still not openable after unlock"
    assert not folder_manager.is_locked(protected), "folder should no longer report as locked"
    assert (protected / "invoice.txt").read_text() == "confidential invoice"
    assert (protected / "sub" / "notes.txt").read_text() == "private notes"
    print(" OK: access restored, nested contents intact and readable")

    print("== registry marks it unlocked but REMEMBERS its credentials ==")
    entry = folder_manager._registry_entry(protected)
    assert entry is not None, "the folder should stay on record after unlocking"
    assert entry.get("locked") is False, "it must no longer be marked locked"
    assert entry.get("profile_id") == PROFILE, (
        "its credentials must be remembered so re-locking reuses the same "
        "password and face instead of silently starting over"
    )
    assert all(e["path"] != str(protected) for e in folder_manager.list_locked_folders())
    print(" OK: marked unlocked, still bound to its own profile")

    print("== failed unlock (wrong key) must NOT downgrade protection ==")
    folder_manager.lock_folder(protected, master_key, PROFILE, progress_cb=lambda m: None)
    wrong_key = security.generate_master_key()
    try:
        folder_manager.unlock_folder(protected, wrong_key, progress_cb=lambda m: None)
        raise SystemExit("FAIL: unlock with the wrong key did not raise")
    except SystemExit:
        raise
    except Exception:

```

```

        pass # expected: authentication/tamper failure
    assert folder_manager.is_locked(protected), "folder must remain locked after a failed unlock"
    if access_control.is_available():
        assert not _can_open(protected), (
            "BUG: a failed unlock attempt left the folder openable – protection was downgraded"
        )
        print(" OK: still locked AND still access-restricted after a failed attempt")
    else:
        print(" OK: still locked after a failed attempt")

    print("== final unlock with the correct key ==")
    folder_manager.unlock_folder(protected, master_key, progress_cb=lambda m: None)
    assert _can_open(protected)
    assert (protected / "invoice.txt").read_text() == "confidential invoice"
    print(" OK: recovered cleanly")

except AssertionError as exc:
    print(f"\nASSERTION FAILED: {exc}")
    _cleanup_and_exit(1)
except SystemExit:
    raise
except Exception as exc:
    print(f"\nUNEXPECTED ERROR: {type(exc).__name__}: {exc}")
    _cleanup_and_exit(1)

print("\nALL ACCESS-CONTROL SELF-TESTS PASSED")
_cleanup_and_exit(0)

```

## \_selftest\_ui.py

*GUI widget construction and preview sizing.*

137 lines

```

"""
Ad-hoc self-test that actually constructs every GUI dialog class with a
real Tkinter root (this machine has a real Windows desktop, so this is not
mocked) and immediately tears each one down. This catches widget-
construction bugs – like passing a pack()-only tuple option (e.g.
pady=(10, 0)) straight into a Frame/Label constructor, which Tk accepts
syntactically but rejects at runtime with TclError: "bad screen distance"
– that headless crypto/logic self-tests can never exercise, since they
never actually build a single widget.

Does NOT require a webcam: AuthDialog/EnrollmentDialog kick off their
camera work in a background thread *after* their widgets are already
built, so we can construct-then-immediately-destroy them without a camera
ever needing to succeed.

Run: python _selftest_ui.py
"""
import shutil
import tempfile
import tkinter as tk
from pathlib import Path

import config

tmp_root = Path(tempfile.mkdtemp(prefix="flock_ui_selftest_"))
config.APP_DATA_DIR = tmp_root / "appdata"
config.VAULT_DIR = config.APP_DATA_DIR / "vault"
config.LOG_DIR = config.APP_DATA_DIR / "logs"
config.DEVICE_KEY_FILE = config.VAULT_DIR / "device.key"
config.FACE_TEMPLATE_FILE = config.VAULT_DIR / "face_template.enc"
config.PASSWORD_VAULT_FILE = config.VAULT_DIR / "password_vault.json"
config.SETTINGS_FILE = config.VAULT_DIR / "settings.json"
config.ensure_dirs()

import ui # noqa: E402

root = tk.Tk()
root.withdraw() # no visible flicker; we're only checking construction succeeds

failures = []

def try_build(label, factory):
    try:
        widget = factory()
        root.update() # process any pending idle tasks the constructor scheduled
        widget.destroy()
        root.update()
        print(f" OK: {label}")

```

```

except Exception as exc:
    failures.append((label, exc))
    print(f"    FAIL: {label} -> {type(exc).__name__}: {exc}")

print("== CameraPanel renders at the intended pixel size, not a tiny collapsed box ==")
import numpy as np # noqa: E402

TOLERANCE_PX = 10 # Label border/padding adds a few px beyond the raw image size

def _assert_reasonable_size(label, w, h, when):
    target_w, target_h = ui.PREVIEW_SIZE
    assert abs(w - target_w) <= TOLERANCE_PX and abs(h - target_h) <= TOLERANCE_PX, (
        f"BUG: preview is {w}x{h} {when} - expected close to {ui.PREVIEW_SIZE} "
        f"(this is exactly the old bug: Label width/height silently switch from "
        f"character-units to pixel-units once an image is set, collapsing the "
        f"preview to a near-invisible size)"
    )

panel = ui.CameraPanel(root)
panel.pack()
root.update_idletasks()
w0, h0 = panel.image_label.winfo_reqwidth(), panel.image_label.winfo_reqheight()
print(f"    before any frame: {w0}x{h0}")
_assert_reasonable_size(panel.image_label, w0, h0, "before any frame")

fake_frame = np.zeros((240, 320, 3), dtype=np.uint8) # arbitrary camera-like BGR frame
panel.set_frame(fake_frame)
root.update_idletasks()
w1, h1 = panel.image_label.winfo_reqwidth(), panel.image_label.winfo_reqheight()
print(f"    after a real frame: {w1}x{h1}")
_assert_reasonable_size(panel.image_label, w1, h1, "after receiving a frame")

panel.set_status("Face recognized")
assert panel.status_label.cget("fg") == ui.STATUS_COLOR_SUCCESS
panel.set_status("Face not recognized")
assert panel.status_label.cget("fg") == ui.STATUS_COLOR_DENIED
panel.destroy()
print(f"    OK: preview stays a visible {ui.PREVIEW_SIZE[0]}x{ui.PREVIEW_SIZE[1]} px before and after a frame, status colors
switch correctly")

print("== constructing every dialog/screen with real Tkinter widgets ==")
try_build(
    "FolderCredentialsWizard",
    lambda: ui.FolderCredentialsWizard(root, "ui-test-profile", tmp_root / "SomeFolder"),
)
try_build(
    "FolderPicker (empty)",
    lambda: ui.FolderPicker(root, [], title="Test", action_label="Go"),
)
try_build(
    "FolderPicker (populated)",
    lambda: ui.FolderPicker(
        root,
        [{"path": str(tmp_root / "A"), "profile_id": "p1", "locked": True, "exists": True},
        {"path": str(tmp_root / "B"), "profile_id": "p2", "locked": False, "exists": False}],
        title="Test", action_label="Go", allow_browse=True,
    ),
)
try_build("LockedFolderPicker", lambda: ui.LockedFolderPicker(root))
try_build("EnrollmentDialog", lambda: ui.EnrollmentDialog(root, "ui-test-profile"))
try_build("NewPasswordDialog", lambda: ui.NewPasswordDialog(root))
try_build("SecuritySettingsDialog", lambda: ui.SecuritySettingsDialog(root))
try_build("AuthDialog (mode=full)", lambda: ui.AuthDialog(root, "ui-test-profile", mode="full"))
try_build("AuthDialog (mode=face_only)", lambda: ui.AuthDialog(root, "ui-test-profile", mode="face_only"))

print("== constructing the main App window itself ==")
try:
    app = ui.App()
    app.update()
    app.destroy()
    print("    OK: App main window")
except Exception as exc:
    failures.append(("App main window", exc))
    print(f"    FAIL: App main window -> {type(exc).__name__}: {exc}")

root.destroy()
shutil.rmtree(tmp_root, ignore_errors=True)

if failures:
    print(f"\n{len(failures)} WIDGET CONSTRUCTION FAILURE(S):")
    for label, exc in failures:
        print(f"    - {label}: {exc}")
    raise SystemExit(1)

```



```
print("\nALL UI WIDGET CONSTRUCTION SELF-TESTS PASSED")
```